

# Contents

|       |                                                   |    |
|-------|---------------------------------------------------|----|
| 1     | 从零构建 MicroClaw .....                              | 19 |
| 1.1   | 第 1 章 什么是<br>MicroClaw：从聊天机器<br>人到执行型 Agent ..... | 19 |
| 1.1.1 | 1.1 从“问答”到“执<br>行” .....                          | 19 |
| 1.1.2 | 1.2 MicroClaw 的<br>目标与非目标 .                       | 22 |
| 1.1.3 | 1.3 项目演进脉络<br>与关键里程碑 .                            | 24 |
| 1.1.4 | 1.4 本章小结 ..                                       | 26 |
| 1.2   | 第 2 章 架构总览：渠道无<br>关核心 + 平台适配器 .                   | 27 |
| 1.2.1 | 2.1 系统分层与主<br>数据流 .....                           | 27 |

|       |                                                  |    |
|-------|--------------------------------------------------|----|
| 1.2.2 | 2.2 AppState 与<br>核心模块装配 .                       | 29 |
| 1.2.3 | 2.3 多渠道一致性<br>与差异化处理 .                           | 31 |
| 1.2.4 | 2.4 本章小结 .                                       | 33 |
| 1.3   | 第 3 章 Agent Loop 全景：<br>一次请求如何完成 . . . .         | 34 |
| 1.3.1 | 3.1 输入归一化与<br>会话恢复 . . . . .                     | 34 |
| 1.3.2 | 3.2 系统提示词构<br>建与上下文压<br>缩 . . . . .              | 36 |
| 1.3.3 | 3.3 tool_use /<br>tool_result 迭代机<br>制 . . . . . | 38 |
| 1.3.4 | 3.4 失败、重试与<br>边界条件 . . . . .                     | 40 |

|       |                                                |    |
|-------|------------------------------------------------|----|
| 1.3.5 | 3.5 本章小结 .                                     | 42 |
| 1.4   | 第 4 章 Agent Loop 深潜：<br>状态机、事件流与可恢复<br>性 ..... | 43 |
| 1.4.1 | 4.1 stop_reason<br>驱动的状态转<br>移 .....           | 43 |
| 1.4.2 | 4.2 流式输出与事<br>件总线 .....                        | 45 |
| 1.4.3 | 4.3 最大迭代上限<br>与反失控设计 .                         | 47 |
| 1.4.4 | 4.4 会话持久化与<br>跨轮次连续执<br>行 .....                | 49 |
| 1.4.5 | 4.5 本章小结 ..                                    | 51 |
| 1.5   | 第 5 章 工具系统设计：统<br>一抽象与可扩展执行 ..                 | 51 |

|       |                                |    |
|-------|--------------------------------|----|
| 1.5.1 | 5.1 Tool Trait 与注册中心 .....     | 51 |
| 1.5.2 | 5.2 内建工具分类与边界 .....            | 53 |
| 1.5.3 | 5.3 子代理工具集裁剪 .....             | 55 |
| 1.5.4 | 5.4 工具开发模板与质量门禁 ...            | 56 |
| 1.5.5 | 5.5 本章小结 .                     | 58 |
| 1.6   | 第 6 章 安全机制：风险分级、审批门与权限模型 ..... | 59 |
| 1.6.1 | 6.1 Tool Risk 与执行策略标签 ...      | 59 |
| 1.6.2 | 6.2 高风险工具双阶段审批 .....           | 60 |

|       |                                           |    |
|-------|-------------------------------------------|----|
| 1.6.3 | 6.3 聊天作用域授权与<br>control_chat .            | 62 |
| 1.6.4 | 6.4 Web Auth 与<br>scopes 设计 ..            | 64 |
| 1.6.5 | 6.5 本章小结 .                                | 65 |
| 1.7   | 第 7 章 Sandbox 机制：<br>隔离执行与失败策略 ..         | 66 |
| 1.7.1 | 7.1 Off / All 模式<br>与运行时路由 .              | 66 |
| 1.7.2 | 7.2<br>DockerSandbox<br>资源与网络约<br>束 ..... | 68 |
| 1.7.3 | 7.3 挂载路径、符<br>号链接与白名单防<br>护 .....         | 69 |

|       |       |                     |
|-------|-------|---------------------|
| 1.7.4 | 7.4   |                     |
|       |       | require_runtime     |
|       |       | 与 fail-open /       |
|       |       | fail-closed .... 71 |
| 1.7.5 | 7.5   | 本章小结 .. 73          |
| 1.8   | 第 8 章 | 记忆系统：文件记            |
|       |       | 忆 + 结构化记忆的分层模       |
|       |       | 型 ..... 74          |
| 1.8.1 | 8.1   | AGENTS.md           |
|       |       | 持久化记忆 ... 74        |
| 1.8.2 | 8.2   | SQLite 结构           |
|       |       | 化记忆与生命周             |
|       |       | 期 ..... 75          |
| 1.8.3 | 8.3   | Reflector 提         |
|       |       | 取与记忆质量闸             |
|       |       | 门 ..... 77          |

|       |                                              |    |
|-------|----------------------------------------------|----|
| 1.8.4 | 8.4 语义检索与<br>token 预算注<br>入 .....            | 78 |
| 1.8.5 | 8.5 本章小结 .                                   | 80 |
| 1.9   | 第 9 章 Todo、计划与执<br>行编排 .....                 | 81 |
| 1.9.1 | 9.1 计划先行的执<br>行哲学 .....                      | 81 |
| 1.9.2 | 9.2 todo_read /<br>todo_write 协议实<br>践 ..... | 83 |
| 1.9.3 | 9.3 多步骤任务的<br>可观测推进 ...                      | 84 |
| 1.9.4 | 9.4 本章小结 .                                   | 86 |
| 1.10  | 第 10 章 定时任务系统：<br>从一次性触发到持续自动<br>化 .....     | 87 |

|        |                                                         |    |
|--------|---------------------------------------------------------|----|
| 1.10.1 | 10.1 调度模型与<br>Cron 约定 . . . .                           | 87 |
| 1.10.2 | 10.2 Scheduler 与<br>Agent Loop 的耦<br>合点 . . . . .       | 88 |
| 1.10.3 | 10.3 DLQ 与任务<br>恢复机制 . . . . .                          | 90 |
| 1.10.4 | 10.4 本章小结 .                                             | 91 |
| 1.11   | 第 11 章 多渠道适配层：<br>Telegram、Discord、<br>Web 的共性与差异 . . . | 92 |
| 1.11.1 | 11.1 渠道路由与消<br>息模型 . . . . .                            | 92 |
| 1.11.2 | 11.2 长消息分片与<br>传输约束 . . . . .                           | 93 |
| 1.11.3 | 11.3 统一核心与适<br>配器扩展策略 .                                 | 94 |



|        |                                            |     |
|--------|--------------------------------------------|-----|
| 1.11.4 | 11.4 本章小结 .                                | 96  |
| 1.12   | 第 12 章 Web 控制面与可<br>观测性 .....              | 97  |
| 1.12.1 | 12.1 Web API 安全<br>面与配置自检 .                | 97  |
| 1.12.2 | 12.2 使用量、记忆<br>与稳定性指标 .                    | 98  |
| 1.12.3 | 12.3 运行手册与<br>SLO 告警闭<br>环 .....           | 100 |
| 1.12.4 | 12.4 本章小结 .                                | 101 |
| 1.13   | 第 13 章 配置体系与设计<br>决策：每个参数背后的工<br>程取舍 ..... | 102 |
| 1.13.1 | 13.1 配置装载、默<br>认值与兼容策<br>略 .....           | 102 |

|        |                                          |     |
|--------|------------------------------------------|-----|
| 1.13.2 | 13.2 关键参数组逐<br>项解读 .....                 | 104 |
| 1.13.3 | 13.3 典型部署配置<br>与风险画像 ...                 | 111 |
| 1.13.4 | 13.4 本章小结 .                              | 114 |
| 1.14   | 第 14 章 从代码到发布：<br>测试、升级、运维与路线<br>图 ..... | 115 |
| 1.14.1 | 14.1 测试基线与稳<br>定性烟测 .....                | 115 |
| 1.14.2 | 14.2 升级路径、回<br>滚策略与发布节<br>奏 .....        | 116 |
| 1.14.3 | 14.3 技术债与下一<br>阶段演进方向 .                  | 118 |
| 1.14.4 | 14.4 本章小<br>结 .....                      | 120 |

|        |                                |     |
|--------|--------------------------------|-----|
| 1.15   | 第 15 章 源码结构化导读：<br>从仓库树到执行路径 . | 121 |
| 1.15.1 | 15.1 仓库分层阅读<br>法 .....         | 121 |
| 1.15.2 | 15.2 crates 级别<br>的职责边界 . .    | 122 |
| 1.15.3 | 15.3 src/ 主模块<br>导读 .....      | 123 |
| 1.15.4 | 15.4 工具模块逐项<br>导读 .....        | 128 |
| 1.15.5 | 15.5 文档与源码联<br>动阅读法 .....      | 131 |
| 1.15.6 | 15.6 阅读与改造的<br>实践清单 .....      | 132 |
| 1.15.7 | 15.7 本章小<br>结 .....            | 133 |

|        |                              |     |
|--------|------------------------------|-----|
| 1.16   | 第 16 章 安全深水区：威胁建模与防护决策 . . . | 134 |
| 1.16.1 | 16.1 资产识别：到底在保护什么 .          | 134 |
| 1.16.2 | 16.2 攻击面清单：从输入到执行 .          | 136 |
| 1.16.3 | 16.3 典型威胁场景与对策 . . . . .     | 137 |
| 1.16.4 | 16.4 防护策略优先级模型 . . . . .     | 140 |
| 1.16.5 | 16.5 安全评审模板 . . . . .        | 141 |
| 1.16.6 | 16.6 安全运营建议 . . . . .        | 142 |
| 1.16.7 | 16.7 本章小结 . . . . .          | 143 |

|        |                                             |     |
|--------|---------------------------------------------|-----|
| 1.17   | 第 17 章 Sandbox 实战与<br>攻防：部署、诊断与演<br>练 ..... | 144 |
| 1.17.1 | 17.1 三种环境的沙<br>箱模板 .....                    | 144 |
| 1.17.2 | 17.2 上线前验证清<br>单 .....                      | 146 |
| 1.17.3 | 17.3 典型故障与排<br>障流程 .....                    | 147 |
| 1.17.4 | 17.4 攻防演练案例<br>(10 例) .....                 | 149 |
| 1.17.5 | 17.5 沙箱策略演进<br>建议 .....                     | 151 |
| 1.17.6 | 17.6 本章小<br>结 .....                         | 152 |

|        |                      |        |
|--------|----------------------|--------|
| 1.18   | 第 18 章 Agent Loop 案例 |        |
|        | 工坊：30 个任务如何收         |        |
|        | 敛 .....              | 152    |
| 1.18.1 | 18.1 使用说明            | .152   |
| 1.18.2 | 18.2 案例集             | .. 153 |
| 1.18.3 | 18.3 模板化执行建          |        |
|        | 议 .....              | 168    |
| 1.18.4 | 18.4 本章小             |        |
|        | 结 .....              | 169    |
| 1.19   | 第 19 章 配置项设计百科：      |        |
|        | 意图、风险与反模式            | . 169  |
| 1.19.1 | 19.1 阅读方式            | .169   |
| 1.19.2 | 19.2 LLM 组           | . 170  |
| 1.19.3 | 19.3 上下文与记忆          |        |
|        | 组 .....              | 173    |
| 1.19.4 | 19.4 路径与隔离           |        |
|        | 组 .....              | 177    |

|         |                                 |     |
|---------|---------------------------------|-----|
| 1.19.5  | 19.5 Sandbox<br>组 .....         | 180 |
| 1.19.6  | 19.6 Web 控制面<br>组 .....         | 185 |
| 1.19.7  | 19.7 渠道与权限<br>组 .....           | 189 |
| 1.19.8  | 19.8 语音、技能与<br>扩展组 .....        | 192 |
| 1.19.9  | 19.9 费用与可观测<br>组 .....          | 194 |
| 1.19.10 | 19.10 配置治理总<br>原则 .....         | 195 |
| 1.19.11 | 19.11 本章小<br>结 .....            | 196 |
| 1.20    | 第 20 章 事故复盘与演练：<br>从告警到恢复 ..... | 197 |

|        |                                              |     |
|--------|----------------------------------------------|-----|
| 1.20.1 | 20.1 复盘方<br>法 .....                          | 197 |
| 1.20.2 | 20.2 事故案例<br>(20 例) .....                    | 198 |
| 1.20.3 | 20.3 演练计划<br>(季度版) ...                       | 209 |
| 1.20.4 | 20.4 复盘文化建<br>议 .....                        | 210 |
| 1.20.5 | 20.5 本章小<br>结 .....                          | 210 |
| 1.21   | 第 21 章 全书总结：把<br>Agent 系统做成工程系<br>统 .....    | 211 |
| 1.22   | 第 22 章 实战练习题库：<br>从入门到进阶的 60 组工<br>程训练 ..... | 214 |



|        |                                              |     |
|--------|----------------------------------------------|-----|
| 1.22.1 | 22.1 使用方<br>法 .....                          | 214 |
| 1.22.2 | 22.2 训练题<br>库 .....                          | 215 |
| 1.22.3 | 22.3 题库使用建<br>议 .....                        | 415 |
| 1.22.4 | 22.4 本章小<br>结 .....                          | 415 |
| 1.23   | 第 23 章 设计决策备忘录：<br>80 条 ADR 风格取舍记<br>录 ..... | 416 |
| 1.23.1 | 23.1 阅读说<br>明 .....                          | 416 |
| 1.23.2 | 23.2 决策记<br>录 .....                          | 417 |
| 1.23.3 | 23.3 决策管理建<br>议 .....                        | 683 |

|        |                                |     |
|--------|--------------------------------|-----|
| 1.23.4 | 23.4 本章小<br>结 .....            | 684 |
| 1.24   | 附录 .....                       | 684 |
| 1.24.1 | 附录 A 术语<br>表 .....             | 684 |
| 1.24.2 | 附录 B 工具清单与<br>风险等级速<br>查 ..... | 688 |
| 1.24.3 | 附录 C 默认配置<br>项速查表 ....         | 693 |
| 1.24.4 | 附录 D 参考资料<br>与代码索引 ..          | 697 |

# 1 从零构建 MicroClaw

## 1.1 第 1 章 为什么是 Micro-Claw：从聊天机器人到执行型 Agent

### 1.1.1 1.1 从“问答”到“执行”

多数聊天机器人在系统设计上遵循同一条短路径：接收用户文本，拼接提示词，调用模型，返回文本。这个模式在问答类场景非常高效，但在工程任务上会很快碰到瓶颈。

第一类瓶颈是“动作缺失”。用户说“帮我查这个仓库里所有超时配置”，仅靠生成文本无法完成真实检索动作；机器人需要能读文件、跑命令、过滤结果。第二类瓶

颈是“过程不可追踪”。用户说“先列计划再执行”，传统 bot 只能输出一个计划文本，无法保证后续执行严格对齐。第三类瓶颈是“状态丢失”。多轮对话中上下文很快过长，系统在压缩时常把关键执行痕迹丢掉，导致下一轮重复劳动。

MicroClaw 的切入点是把“聊天界面”视为任务入口，把“执行系统”视为产品核心。它不是让模型在文本里扮演 shell，而是把 shell、文件系统、网页检索、调度器、记忆系统做成明确的工具接口，再由 Agent Loop 驱动这些接口完成闭环。

这件事的价值不在“能调用多少工具”，而在“能否稳定完成任务”。稳定完成任务至少意味着四点：

1. 行为可解释：每次执行都有可检查的工具调用路径。
2. 状态可恢复：服务重启后，会话与任务不丢失。
3. 风险可控制：高风险工具有审批和策略边界。
4. 结果可复盘：失败时能知道失败在哪一层。

因此，MicroClaw 不是在“增强聊天”，而是在“重写执行面”。这也是后文会反复出现的一条主线：模型负责决策，运行时负责约束与落地。

## 1.1.2 1.2 MicroClaw 的目标与非目标

从代码和文档可以看到，项目在目标定义上相对克制。它追求的是工程可用，而不是研究型“全能体”。

核心目标可归纳为五类：

1. 多渠道一致执行：Telegram、Discord、Web 共享同一 Agent Loop。
2. 工具优先而非文本优先：需要执行时必须通过工具，不靠“描述式完成”。
3. 状态持久化：消息、会话、任务、记忆都落盘，可续跑。
4. 可运维：有 runbook、有 SLO 指标、有自检接口。

5. 可演进：通过工具和技能扩展能力，而不是分叉执行核心。

对应地，MicroClaw 也明确了非目标：

1. 不是通用低代码平台：不提供大而全的可视化流程编排器。
2. 不是多租户企业权限系统：当前认证授权在逐步完善中（见 RFC）。
3. 不是“零风险自动执行器”：默认配置仍强调上手成本和可用性平衡。

这个目标/非目标边界很关键。很多 Agent 项目失败，不是技术做不到，而是目标过宽、边界含糊，最终在体验、安全、可维护性上同时

失分。MicroClaw 的策略是“先做少而硬的核心”，例如围绕 Agent Loop、工具运行时、记忆体系和调度器建立主干，再逐层补控制面和治理能力。

### 1.1.3 1.3 项目演进脉络与关键里程碑

从 website/blog 与 docs 的材料看，项目演进大致经历了三个阶段。

阶段一：可用性优先。初期重点是把“聊天入口 + 工具执行 + 基础持久化”打通，让系统真正可跑。这个阶段常见设计是默认路径尽量短，例如沙箱默认关闭、配置尽量少，以降低首次部署摩擦。



阶段二：可靠性优先。随着功能增多，系统开始补齐长期运行能力：会话恢复、上下文压缩、调度任务、失败记录、DLQ、运行报告。这里的关键不是“新增一个功能”，而是把功能放进同一稳定闭环。

阶段三：治理与平台化优先。近期提交和 RFC 显示，项目进入“可控扩展”阶段：Auth scopes、Hooks 事件模型、配置自检、指标统一命名、升级指南等都在收敛。换句话说，系统开始为规模化使用预留规则空间。

一个值得注意的取舍是：项目没有采用“每个渠道独立智能体”的常见方式，而是坚持共享核心。这样

做让开发者在新增渠道时主要处理 I/O 适配，不需要复制 Agent Loop，避免策略漂移。

另一个关键里程碑是“结构化记忆 + Reflector”。这意味着系统从“把记忆写在文件里”升级为“可检索、可去重、可追踪”的记忆层，并且通过后台任务持续维护质量。这种能力在长周期任务里非常重要，因为用户真正关心的是“下周它还记得并做对”。

#### **1.1.4 1.4 本章小结**

本章建立了全书起点：MicroClaw 的核心不是聊天，而是执行；它的价值在于把模型决策嵌入一个有状态、有边界、可恢复的运行时。

下一章将进入系统结构层，解释“渠道无关核心 + 平台适配器”如何在代码中落地，以及这个分层如何直接影响后续安全、扩展和运维成本。

## 1.2 第2章 架构总览：渠道无关核心 + 平台适配器

### 1.2.1 2.1 系统分层与主数据流

MicroClaw 的总体结构可以概括为四层：接入层、执行层、状态层、后台层。

接入层负责“进出站协议”，对应 `src/channels/*` 与 `src/web.rs`。它处理不同平台的消息事件、格式约束、身份字段和回包方

式。执行层是核心，主要在 `src/agent_engine.rs`，负责循环调用模型与工具。状态层由存储和配置组成，包括会话、消息、任务、记忆、指标。后台层包括 `scheduler` 与 `reflector`，处理定时与记忆维护。

这四层之间的数据流遵循一个稳定路径：

1. 渠道收到用户事件，归一化为内部请求。
2. 请求进入共享 Agent Loop。
3. Agent Loop 按需调用 Tool-Registry。
4. 工具执行结果回流到 Agent Loop，直到模型结束。

5. 结果通过渠道适配器发回, 并落库。
6. 后台任务周期性读取状态层, 触发下一轮执行或维护。

该路径的重要点在于“统一核心, 不统一外壳”。也就是说, 渠道差异被隔离在入口出口, 任务逻辑尽量统一在执行层。这直接降低了多平台维护成本。

### 1.2.2 2.2 AppState 与核心模块装配

在运行时层面, MicroClaw 通过一个集中状态容器把关键能力装配到一起。虽然不同模块位于不同 crate 或目录, 但调用关系收敛到统一状态对象。

这个装配思路有三个工程优势：

1. 依赖关系可见：LLM、DB、工具注册、渠道注册、记忆服务、hooks 都在同一图里。
2. 生命周期清晰：启动阶段初始化，运行阶段共享，退出阶段可控回收。
3. 测试边界明确：单元测试可替换局部模块，集成测试可走完整链路。

例如工具系统初始化时，需要配置 `working_dir`、`working_dir_isolation`、`sandbox` 路由器、数据库句柄和渠道注册器。把这些依赖显式注入到 `Tool-Registry`，比在工具内部“隐式读取

全局单例”更容易做策略控制和错误定位。

另一个装配细节是“默认值落点”。配置默认值集中定义，文档又由脚本生成，减少“代码默认值”和“文档默认值”漂移。对于运行时项目，这类一致性比表面功能更重要，因为运维事故常常来自默认行为误判。

### 1.2.3 2.3 多渠道一致性与差异化处理

“多渠道一致性”不等于“所有渠道行为一模一样”。MicroClaw 的做法是把一致性放在语义层，把差异化放在传输层。

语义层一致性主要体现在：

1. 同一用户请求都走相同 Agent Loop。
2. 同一工具调用语义一致。
3. 同一权限模型（当前聊天 vs control chat）一致。
4. 同一会话持久化与恢复机制一致。

传输层差异化主要体现在：

1. 平台消息长度上限不同，需要分片策略。
2. 平台的附件、提及、线程模型不同。
3. Web 存在额外的认证、速率和并发控制。

这种分工在代码里有明确体现：新增平台时，文档明确建议复



用 `process_with_agent`，不要新建平台专属 `loop`。这个约束的价值是防止“同一产品，不同平台，不同脑子”。

当系统规模变大后，这种约束尤为关键。若平台各自演进，很快会出现策略不一致：某平台有审批、某平台没有；某平台支持会话恢复、某平台丢上下文。MicroClaw 通过共享核心提前规避了这个结构性问题。

#### 1.2.4 2.4 本章小结

本章给出了 MicroClaw 的系统地图：入口适配、共享执行、持久化状态、后台维护四层协作，核心原则是“语义统一，传输分离”。

下一章将从宏观进入微观，详细拆解一次请求在 Agent Loop 中的完整生命周期，并解释为什么这个循环是全系统最关键的设计支点。

## **1.3 第 3 章 Agent Loop 全景：一次请求如何完成**

### **1.3.1 3.1 输入归一化与会话恢复**

在 MicroClaw 中，“一条消息”的处理不是一次模型调用，而是一段状态驱动流程。入口可能来自 Telegram、Discord 或 Web，但进入 `process_with_agent_impl` 后，都会变成同一种内部语义：带有渠道、`chat_id`、`chat_type` 的请求上下文。

流程起点有一个容易被忽视但很关键的分支：显式记忆快速路径。代码会先尝试识别用户是否在做明确“记住某事实”的请求，如果满足条件，系统可直接落结构化记忆并返回确认，而不必启动完整工具循环。这个设计优化了两件事：

1. 降低高频记忆写入的延迟和成本。
2. 避免把简单记忆操作交给复杂循环，减少不确定性。

随后，系统进入会话恢复逻辑。它优先尝试从 session 存储读取历史消息流；如果 session 不存在或损坏，则回退到数据库历史重建。重建时群聊与私聊采用不同取数策

略：群聊倾向“上次 bot 回复之后”的窗口，私聊倾向“最近消息窗口”。

这一步体现了 Agent 系统的重要原则：上下文不是“越多越好”，而是“与当前任务相关且结构完整”。MicroClaw 不盲目拼接全量历史，而是通过 session + 回退策略在完整性和成本之间取平衡。

### 1.3.2 3.2 系统提示词构建与上下文压缩

会话消息准备完成后，Agent Loop 会组装系统提示词。该过程不是一个静态模板，而是多源上下文拼接：

1. 机器人身份与渠道信息。

2. 文件记忆 (AGENTS.md)。
3. 结构化记忆注入片段。
4. 技能目录与可激活能力。
5. 可选 SOUL 定制内容。

其中“结构化记忆注入”最具工程含量。系统会对记忆进行相关性排序，并受 `memory_token_budget` 控制，超预算内容会被截断并记录“省略计数”。这不是简单裁剪，而是可观测裁剪：系统会把注入方式、候选数、命中数、估算 token 记录下来，便于后续诊断“为什么这轮没想起某条记忆”。

如果消息过长超过 `max_session_messages`，系统会触发上下文压缩：先归档，再保留近

期关键消息，避免无限增长。压缩不只是成本优化，也是正确性防线。没有压缩的长对话最终会把关键指令淹没，导致 agent 看似“记住很多”，实则“做错更多”。

### 1.3.3 3.3 tool\_use / tool\_result 迭代机制

进入主循环后，系统最多执行 `max_tool_iterations`（默认 100）轮。每轮流程可概括为：

1. 触发 `BeforeLLMCall` hooks（可放行、阻断、补丁）。
2. 调用 LLM（可流式或非流式）。
3. 根据 `stop_reason` 分支处理。

如果 `stop_reason` 为 `end_turn` 或 `max_tokens`，系统聚合文本块并形

成最终回复；如果为 `tool_use`，则把模型返回的工具调用逐个执行，再把 `tool_result` 回灌给模型，进入下一轮。

在 `tool_use` 分支里，MicroClaw 做了四层保护：

1. 执行策略检查：先看工具 `policy` 是否允许在当前 `sandbox` 状态下执行。
2. 风险审批检查：高风险工具先走审批门。
3. Hook 前后处理：`BeforeToolCall` 可改输入，`AfterToolCall` 可改输出或阻断。

4. 失败记录: 非审批型失败会加入 `failed_tools` 集合, 最终回包附带执行注记。

这套机制把“工具调用”从模型幻觉风险区转为运行时可控区。模型决定“做什么”, 运行时决定“能不能做、怎么做、失败怎么记”。

### 1.3.4 3.4 失败、重试与边界条件

Agent Loop 的稳定性取决于它如何处理“非理想输出”。MicroClaw 在这方面有多个具体策略。

第一, 空可见回复重试。模型可能输出思考块却没有可见文本; 系统会自动注入 `runtime guard`, 要求模型再回答一次, 并且只重试一次, 避免无限重试。



第二，高风险审批自动二次调用。工具返回 `approval_required` 时，系统按约定自动重试一次，以完成双阶段确认流程。这使审批行为对用户尽量透明，同时仍保留安全门槛。

第三，最大迭代保护。当达到上限时，系统不会让会话停在 `tool_result` 状态，而是补一条 `assistant` 消息收口，避免下次恢复时出现“结果不跟随调用”的协议错误。

第四，未知 `stop_reason` 兜底。即使模型返回非预期 `stop_reason`，系统也会尽量落 `session` 并返回可读文本，优先保证连续性。

第五，工具失败可见性。请求最终成功时，如果途中有工具失败，系统会在结尾附加 execution note。这一小设计避免了“看起来回答正常，但执行其实失败”的隐性错误。

### 1.3.5 3.5 本章小结

本章展示了 Agent Loop 的完整主干：准备上下文、调用模型、执行工具、回灌结果、持久化状态。它并不是“模型外包器”，而是一个带策略、带容错、带可恢复性的执行状态机。

下一章将继续深潜 Agent Loop 的内部结构，重点讨论 stop\_reason 状态转移、流式事件、恢复语义以及循环边界的工程设计。

## 1.4 第 4 章 Agent Loop 深潜：状态机、事件流与可恢复性

### 1.4.1 4.1 stop\_reason 驱动的状态转移

MicroClaw 的 Agent Loop 可以视为一个由 stop\_reason 驱动的有限状态机。核心状态并不多，但边界定义明确。

主要状态可抽象为：

1. LLM\_CALLING：向模型请求下一步。
2. TOOL\_EXECUTING：执行模型请求的工具。

3. FINALIZING: 收敛输出并持久化会话。
4. ABORTED: 被策略阻断或达到上限后终止。

`stop_reason` 是状态机的转移条件:

1. `tool_use`: 从 `LLM_CALLING` 转入 `TOOL_EXECUTING`。
2. `end_turn`: 从 `LLM_CALLING` 进入 `FINALIZING`。
3. `max_tokens`: 进入受限 `FINALIZING`, 并可能触发提示性收尾。
4. 其他值: 走未知分支兜底, 仍尝试进入 `FINALIZING`。

这种显式转移的好处是:

1. 每一轮都有边界, 循环不会凭直觉推进。
2. 错误分支可单独观测和测试。
3. 新增 `stop_reason` 或 `provider` 行为变化时, 可定位影响面。

在多 `provider` 场景下, 这种状态机设计尤其重要。不同模型厂商对 `stop_reason` 的细节约定可能不同, 统一状态转移层可以消化兼容差异, 避免业务逻辑散落在 `provider` 适配代码中。

### 1.4.2 4.2 流式输出与事件总线

Agent Loop 不仅产生最终文本, 还会产生过程事件。AgentEvent 结构显示系统把执行过程拆成可流式消费的事件流: 迭代开始、工具

开始、工具结果、文本增量、最终响应。

这种设计带来三个直接收益：

1. UI 体验：前端可实时显示“第几轮、在跑什么工具、工具是否报错”。
2. 诊断能力：出问题时无用猜，能看到故障落点。
3. 扩展可能：未来可把事件流接入审计或智能回放系统。

事件模型本质上是“把内部状态公开为稳定接口”。只要事件契约稳定，展示层和控制层就可以独立演进。比如 Web 端可以做 timeline，命令行可以做简化进度条，二者不

需要修改 Agent Loop 的核心流程。

另一个细节是流式 delta 的转发逻辑。系统在流式模式下将模型文本增量转发到统一事件通道，而非直接耦合具体 UI。这个隔离让“输出机制”与“展示机制”解耦，有利于后续接入更多客户端。

### 1.4.3 4.3 最大迭代上限与反失控设计

Agent 系统最常见的生产事故之一是“循环失控”：模型持续调用工具却无法收敛。MicroClaw 通过多层约束降低该风险。

第一层是硬上限：  
`max_tool_iterations`。达到上限即收口，避免资源持续消耗。

第二层是语义收口：达到上限后系统会补一条 `assistant` 消息，明确说明因迭代上限中止，并建议用户拆分任务。这个动作不仅是用户提示，更是协议稳定措施，确保会话可继续恢复。

第三层是失败反馈：`failed_tools` 集合把本轮失败工具名汇总到最终响应中，让用户知道“任务没完全成功”的具体原因。

第四层是空回复防护：可见内容为空时自动注入 `guard` 并重试一次，



减少“模型看似回答了，用户却什么也没看到”的体验故障。

这些设计共同体现一个思想：系统不追求“永不失败”，而追求“可控失败、可恢复失败、可解释失败”。

#### 1.4.4 4.4 会话持久化与跨轮次连续执行

MicroClaw 的可恢复性建立在“每轮落盘”而非“最终落盘”思路。关键路径包括：

1. 每次得到可持久化 assistant 内容后写入 session。
2. 恢复时优先加载 session，再补齐新用户消息。
3. session 不可用时回退到数据库历史。

这套机制解决了两个现实问题：

1. 进程重启后不丢执行上下文。
2. 适配器短时故障不会导致对话完全断层。

此外，系统会在会话持久化前剥离图像块等非必要重量数据，避免 session 膨胀过快。这体现了“可恢复性”和“成本控制”之间的平衡。

从产品视角看，可恢复性直接决定用户信任。用户在第 10 轮交给 agent 的任务，如果第 11 轮因为重启而归零，任何“智能”都失去价值。MicroClaw 把会话恢复放进核心路径，正是基于这个判断。

### **1.4.5 4.5 本章小结**

本章把 Agent Loop 从“流程描述”升级为“状态机理解”：状态转移由 stop\_reason 驱动，事件流暴露过程信息，边界保护限制失控，会话持久化支撑连续执行。

下一章将切换到工具系统，解释 Agent Loop 如何通过统一工具抽象连接真实世界动作，并保持扩展能力与策略一致性。

## **1.5 第5章 工具系统设计：统一抽象与可扩展执行**

### **1.5.1 5.1 Tool Trait 与注册中心**

MicroClaw 的工具系统从一个简单接口出发：每个工具都实现统一

Tool trait, 至少包含名称、定义 (JSON Schema) 和执行函数。这个抽象看似普通, 但它把“能力声明”和“执行实现”绑定在同一对象上, 使模型可调用性与运行时可执行性保持一致。

在注册层, ToolRegistry 负责三件事:

1. 构建工具实例与依赖注入。
2. 缓存工具定义, 供模型端声明工具集。
3. 执行时分发到具体工具并补全结果元信息。

结果元信息 (如 duration\_ms、bytes、status\_code、error\_type) 不是锦上添花, 而是诊断基线。没

有这些字段，就难以在运维上区分“工具慢”“工具错”“工具被策略拦截”三类问题。

### 1.5.2 5.2 内建工具分类与边界

根据当前代码与生成文档，内建工具约 29 个，可按职责分为六组：

1. 执 行 与 文 件 组：  
bash、read\_file、write\_file、  
edit\_file、glob、grep。
2. 记 忆 组：  
read\_memory、write\_memory、  
structured\_memory\_\*。
3. 外 部 信 息 组：web\_search、  
web\_fetch、browser。
4. 协 作 组：send\_message、  
export\_chat。

- 5. 调 度 组 : `schedule_task`、`list/pause/resume/cancel/get_task_history`、DLQ 相关。
- 6. 编排与扩展组 : `todo_read`、`todo_write`、`activate_skill`、`sync_skills`、`sub_agent`。

这套分组反映了一个重要设计原则：让工具名直接表达意图边界。比如 `structured_memory_delete` 明确是结构化层操作，不与文件记忆混用；`sub_agent` 明确是委派语义，不与普通工具执行混淆。

命名清晰带来的收益是“模型更容易选对工具，开发者更容易排错”。在 Agent 系统里，这一点通常比单纯增加工具数量更重要。

### 1.5.3 5.3 子代理工具集裁剪

`sub_agent` 是 `MicroClaw` 的一个关键能力：把某些子任务交给受限工具集并行处理。代码中的 `ToolRegistry::new_sub_agent` 明确去掉了高副作用工具，如 `send_message`、`write_memory`、调度与导出等。

这种裁剪有三层意义：

1. 权限最小化：子代理只能做分析和局部执行，不直接操作外部沟通面。
2. 递归防失控：子代理不能再调用 `sub_agent`，避免无限分叉。

3. 结果可控: 子代理输出最终仍回到主代理, 由主代理决定如何对用户呈现。

并行子代理经常被误用为“让系统更聪明”的捷径。正确用法应是“让系统更可分工”: 把独立子问题切出去, 但保留主流程的一致约束。

#### 1.5.4 5.4 工具开发模板与质量门禁

一个新工具从概念到落地, 至少应经过以下步骤:

1. 定义名称与输入 JSON Schema。
2. 实现执行逻辑, 保证返回统一 ToolResult。



3. 在注册中心接入并注入所需依赖。
4. 标定风险等级与执行策略。
5. 编写单元测试与最小集成验证。
6. 更新生成文档，避免文档漂移。

其中第四步非常关键。很多项目把风险策略写在文档里而不是运行时里，最终策略失效。Micro-Claw 的做法是把策略校验放到 `execute_with_auth` 主路径，确保所有工具调用都会经过同一检查。

工具质量门禁应关注四类失败：

1. 参数缺失或类型错误。
2. 权限不足（跨 chat 操作、非 control chat）。

3. 策略拦截 (execution policy / approval)。
4. 运行时异常 (超时、IO、外部依赖失败)。

只有这四类失败可区分，Agent 才能在回复中给出正确后续动作（重试、改参数、申请权限、人工介入）。

### 1.5.5 5.5 本章小结

本章说明了工具系统为何是 Agent runtime 的“执行关节”：统一接口让能力可声明、可执行、可治理；注册中心让策略与观测集中生效；子代理裁剪让并行能力不失控。

下一章进入安全机制，详细讨论风险分级、审批门和授权模型如何在工具路径上形成多层防护。

## 1.6 第6章 安全机制：风险分级、审批门与权限模型

### 1.6.1 6.1 Tool Risk 与执行策略标签

MicroClaw 的安全设计并不是“一刀切禁用高权限”，而是分层控制。第一层是工具风险分级 (low/medium/high)，第二层是执行策略标签 (host-only/sandbox-only/dual)。

风险分级回答“这件事危险吗”，执行策略回答“这件事在哪儿执行”。二者组合后，运行时才能做出具体判断。

例如：

1. `bash` 被标记为 `high`，执行策略为 `dual`。
2. `write_file` / `edit_file` 属于 `medium`，策略为 `host-only`（当前基线）。
3. 大多数读取类工具属于 `low`。

这种标注让系统具备可进化空间。未来如果某工具风险上升，只需要调整策略映射和校验逻辑，不必重写整条执行链。

### 1.6.2 6.2 高风险工具双阶段审批

对高风险工具，`MicroClaw` 在特定上下文（`Web` 或 `control chat`）采用审批门机制。调用路径中，工具首次触发会返回

approval\_required, 随后执行器自动重试一次完成确认流程。

这套机制有几个实践价值：

1. 把“确认”变成运行时协议, 而不是提示词约束。
2. 保留人机协作中的安全摩擦, 不让高风险动作静默通过。
3. 把审批结果结构化, 便于日志与指标统计。

审批门机制还解决了一个现实问题：模型可能会“误触”高风险动作。即便模型决策错误，审批层也能提供最后一道刹车。

需要强调的是，审批不是万能。它能降低误操作概率，但不能替代隔

离执行。真正高风险部署仍需要和 Sandbox、路径防护、运维策略配合使用。

### 1.6.3 6.3 聊天作用域授权与 control\_chat

MicroClaw 在工具输入里注入认证上下文，核心字段包括 caller\_channel、caller\_chat\_id、control\_chat\_ids。随后通过统一授权函数决定目标 chat 是否可操作。

默认规则非常直接：

1. 普通 chat 只能操作自己的 chat\_id。
2. control chat 可以跨 chat 操作。

这条规则覆盖了多个高价值动作：  
send\_message、定时任务管理、导出聊天、todo、全局记忆等。统一规则带来的好处是：

1. 权限逻辑集中，不在每个工具里重复造轮子。
2. 权限失败返回可解释错误，不是静默失败。
3. 测试矩阵清晰，可做系统级权限回归测试。

从系统治理角度看，chat scope 授权是“最小可行 RBAC”。它还不复杂，但已经能解决跨会话误写、错误广播、越权调度等高概率风险。

## 1.6.4 6.4 Web Auth 与 scopes 设计

在 Web 控制面，安全问题从“工具操作”扩展到“管理操作”。RFC 0001 给出了分阶段方案：从单一 token 走向 session cookie + API key + scopes。

当前设计里的 scopes 包括：

1. operator.read：只读查看。
2. operator.write：可执行变更动作。
3. operator.admin：管理级权限。
4. operator.approvals：高风险审批处理。

这组 scopes 的价值在于“把权限从身份抽离为能力”。同一个身



份在不同集成场景下可以发不同 scope 的 key，降低密钥泄露后的爆炸半径。

配套的 config/self\_check 接口进一步把风险显性化：它会检查沙箱状态、认证配置、速率限制参数、hooks 和 OTLP 配置等，并输出 warning 列表。这相当于把安全审查从“文档建议”升级为“运行时可执行检查”。

### 1.6.5 6.5 本章小结

本章讨论了 MicroClaw 的四条安全主线：风险分级、审批门、chat scope 授权、Web scopes。它们共同构成“可执行、可观测、可演进”的安全底座。

下一章将专门分析 Sandbox 机制：当工具真的执行时，系统如何在宿主与容器之间做策略路由，以及如何处理运行时不可用的现实问题。

## 1.7 第 7 章 Sandbox 机制：隔离执行与失败策略

### 1.7.1 7.1 Off / All 模式与运行时路由

MicroClaw 的沙箱配置核心是 `sandbox.mode`，当前有两个主模式：`off` 与 `all`。

1. `off`：直接在宿主执行命令。
2. `all`：优先通过沙箱后端执行（当前主后端为 Docker）。

路由逻辑由 SandboxRouter 统一处理。它根据模式和后端可用性决定实际执行路径，而不是让各工具自行判断。这种集中式路由有两个优势：

1. 策略一致，避免工具间行为分裂。
2. 观测统一，便于统计 fallback 与失败原因。

默认 off 的原因并非忽视安全，而是权衡首次部署摩擦。项目文档也明确建议：生产或高风险场景应启用沙箱并配合白名单。

## 1.7.2 7.2 DockerSandbox 资源与网络约束

在 all 模式下，若 Docker 可用，命令会进入容器执行。创建容器时系统做了多项限制：

1. `--cap-drop ALL` 去掉额外 Linux capabilities。
2. `--security-opt no-new-privileges` 阻止进程提权。
3. 默认可配置 `--network=none` 关闭网络。
4. 支持 `memory_limit`、`cpu_quota`、`pids_limit` 等资源约束。

这一组合体现了“默认限制、按需放开”的思路。对 Agent 执行环境来说，资源限制并不只是成本控

制，也是在降低命令失控时的影响面。

容器命名采用固定前缀 + session key 片段，方便复用和诊断；执行时通过 `docker exec` 并可指定工作目录。超时由统一 timeout 机制控制，超时会返回结构化错误而不是阻塞主流程。

### 1.7.3 7.3 挂载路径、符号链接与白名单防护

Sandbox 的高风险点之一是“挂载路径”。如果挂载了敏感目录，容器隔离价值会显著下降。为此，MicroClaw 对 mount path 做了多层校验：

1. 拒绝包含符号链接组件的路径。

2. 拒绝敏感目录组件（如 `.ssh`、`.aws`、`.gnupg` 等）。
3. 拒绝敏感文件名片段（如 `.env`、私钥类文件名）。
4. 支持外部 `allowlist` 文件，只允许挂载在白名单根路径内。

如果校验失败，系统会记录 `warning` 并回退到原始工作目录路径（兼容路径）。这是一种工程化妥协：尽量保证可运行，同时把风险显性化。对于生产环境，更推荐显式配置 `allowlist` 并将异常视为阻断条件。

这种校验不仅保护容器执行，也保护“误配置”场景。很多安全事故来

自配置错误而非攻击；路径校验能在错误进入生产前给出明确信号。

#### 1.7.4 7.4 `require_runtime` 与 `fail-open` / `fail-closed`

现实中最常见的问题是：配置开了沙箱，但目标机器没有可用 Docker runtime。MicroClaw 通过 `require_runtime` 明确两种行为：

1. `require_runtime = false`: 告警后回退宿主执行 (`fail-open`)。
2. `require_runtime = true`: 直接报错中止 (`fail-closed`)。

这不是“哪个更好”的问题，而是“业务容忍度”问题：

1. 个人环境和开发环境通常更重可用性，可先 fail-open。
2. 生产或高风险自动化场景更重边界一致，应 prefer fail-closed。

更关键的是，系统把该行为做成显式配置，并在自检与文档中明确提示。这避免了“用户以为在沙箱，实际在宿主”的隐性风险。

推荐实践是把 sandbox 策略与环境分层绑定：

1. 本地开发：  
mode=off 或 mode=all +  
require\_runtime=false。



2. 测试环境： mode=all + require\_runtime=true, 验证工具兼容性。
3. 生产环境： mode=all + require\_runtime=true + allowlist + 资源限制。

### 1.7.5 7.5 本章小结

Sandbox 章节的核心结论是：隔离不只是“开关”，而是一组策略组合。MicroClaw 通过路由器、容器约束、路径校验和 runtime 失败策略，把“可选隔离”逐步升级为“可治理隔离”。

下一章将进入记忆系统，讨论长期状态如何被提取、筛选、注入并保持质量稳定。

## 1.8 第8章 记忆系统：文件记忆 + 结构化记忆的分层模型

### 1.8.1 8.1 AGENTS.md 持久化记忆

MicroClaw 的记忆设计不是单存储方案，而是双层模型。第一层是文件记忆：AGENTS.md。它分为全局和每聊天两个作用域，读取直观、编辑简单、可被人直接审阅。

文件记忆层的价值主要有三点：

1. 可读性高：人类可以直接打开查看。
2. 可移植性强：迁移或备份成本低。

3. 可解释性好：用户能明确知道“系统记住了什么”。

但仅有文件记忆并不足以支撑大规模长期任务，因为它在检索效率、冲突管理、生命周期管理上存在天然局限。这也是结构化记忆层出现的原因。

### 1.8.2 8.2 SQLite 结构化记忆与生命周期

结构化记忆存储在数据库 `memories` 表，并伴随类别、来源、置信度、归档状态等元数据。相比纯文本记忆，它支持更细粒度的治理。

关键能力包括：

1. 分类存储：PROFILE、KNOWLEDGE、EVENT。
2. 置信度管理：支持不同来源的可信度区分。
3. 替换链路：通过 supersede 关系管理“旧事实被新事实覆盖”。
4. 生命周期：支持软归档而非简单硬删除。

这种结构允许系统回答更复杂问题：

1. 这条记忆是谁、何时、基于什么写入的？
2. 这条事实是否已过时并被替换？
3. 当前注入给模型的是活动记忆还是低置信度记忆？

换言之，结构化层把“记住”变成“可治理的长期知识状态”。

### 1.8.3 8.3 Reflector 提取与记忆质量闸门

MicroClaw 通过后台 Reflector 周期性从对话中提取长期事实。它不是简单摘要器，而是带规则约束的提取器。

提取规则强调：

1. 只抽取耐久事实，不记闲聊。
2. 输出结构化 JSON，便于后处理。
3. 对冲突事实可标记 `supersedes_id`。
4. 控制每条内容长度和具体性。

更重要的是记忆污染防护。代码里显式避免把“错误行为描述”写成永久事实，例如“工具调用坏了”这类描述可能反向污染后续行为。系统更鼓励把问题写成纠正型 action item（例如 TODO: ensure ...）。

这是一种很实用的经验：长期记忆不应存放“故障复述”，应存放“行为准则”。否则模型读取后可能强化错误模式。

#### 1.8.4 8.4 语义检索与 token 预算注入

记忆最终价值体现在“注入阶段”。MicroClaw 在构建系统提示词时，

会根据当前 query 选择相关记忆，并受 `memory_token_budget` 限制。

注入策略分两档：

1. 语义检索（启用 `sqlite-vec` + `embedding` 时）。
2. 关键词/相关度排序（默认回退路径）。

无论哪种路径，系统都遵循预算上限并记录注入日志（候选数量、选中数量、省略数量、估算 token）。这为调优提供了事实依据。

常见调优实践：

1. 若答非所问：检查 query 截断长度与相关度排序。

2. 若遗忘关键事实：提高 memory\_token\_budget 或改进记忆内容颗粒度。
3. 若注入过重：降低预算并增强分类过滤规则。

记忆注入本质上是检索系统，不是文件拼接。只有可检索、可预算、可观测，记忆才能在长期任务里稳定发挥价值。

### 1.8.5 8.5 本章小结

MicroClaw 的记忆系统用“文件层 + 结构化层 + 反思层 + 注入层”形成闭环：可读、可管、可提取、可利用。



下一章将讨论 Todo 编排机制，说明系统如何把“多步骤任务”从口头计划变成可追踪执行。

## 1.9 第 9 章 Todo、计划与执行编排

### 1.9.1 9.1 计划先行的执行哲学

Agent 系统中最常见的失败不是“不会调用工具”，而是“不会分阶段执行”。用户给出复杂目标后，模型容易直接跳到局部动作，导致路径混乱、回滚困难、进度不可见。

MicroClaw 在系统提示中明确要求：多步骤任务先 `todo_write` 再执行。这条规则把“思考过程”转化

为“外显计划状态”，让执行链条具备同步能力。

计划先行的价值体现在三方面：

1. 用户可见：任务不是黑箱，用户知道系统在做什么。
2. 机器可控：每步有状态（ pending/in\_progress/completed），可验证。
3. 故障可恢复：中断后可从 todo 状态重建执行位置。

从工程管理视角，这相当于把“单次推理问题”转成“状态机推进问题”。前者难测，后者可测。

## 1.9.2 9.2 todo\_read / todo\_write 协议实践

Todo 工具看似简单，但协议设计很关键。系统约束“同一时刻仅一个 in\_progress”，并要求每个阶段完成后同步更新。这避免了计划漂移。

一个推荐执行节奏是：

1. 拆分目标为有限步骤，优先按依赖顺序。
2. 写入 todo，设置第一步 in\_progress。
3. 执行该步所需工具。
4. 完成后更新 todo，把下一步置为 in\_progress。
5. 重复直到全部完成。

在这个节奏下，todo 成为“任务真相源”。任何输出都应与此 todo 当前状态一致，避免“说完成了但状态未更新”的管理漏洞。

此外，todo 数据落在 chat 作用域，天然继承权限边界。普通 chat 无法跨 chat 操作他人计划，control chat 才有跨域能力。这样可以在编排能力提升的同时，保持作用域安全。

### 1.9.3 9.3 多步骤任务的可观测推进

Todo 与 Agent Loop 结合后，系统可以构建可观测执行轨迹：

1. 计划层：步骤状态变化。
2. 执行层：工具调用日志与结果。

3. 输出层：中间消息与最终总结。

这三层轨迹能够回答关键运维问题：

1. 任务卡在哪一步？
2. 卡住是策略拒绝、权限问题还是运行失败？
3. 失败后是否重试过，是否有替代路径？

在真实团队协作中，可观测推进比“最终回答质量”更重要。因为复杂任务很少一次做对，关键是能否快速定位偏差并继续推进。

MicroClaw 的“计划先行”并不是限制模型创造力，而是给创造力加上执行骨架。没有骨架，长任务会

变成一次次即兴发挥；有骨架，系统才具备工程可依赖性。

### 1.9.4 9.4 本章小结

Todo 机制把复杂任务执行从“隐性思维链”升级为“显性状态链”。它提升了可见性、可恢复性和协作效率，是 Agent 走向生产可用的关键一步。

下一章将继续讨论自动化能力，聚焦 Scheduler 如何在无人工触发场景下稳定运行 Agent Loop。

## 1.10 第 10 章 定时任务系统： 从一次性触发到持续自 动化

### 1.10.1 10.1 调度模型与 Cron 约定

MicroClaw 的调度能力由一组工具接口和后台循环共同完成。前台通过 `schedule_task` 等工具声明任务，后台 `scheduler` 每分钟轮询到期任务并触发执行。

系统支持两类计划：

1. `once`：一次性执行，使用 ISO 时间戳。
2. `cron`：周期执行，采用 6 字段（含秒）格式。

这个 6 字段约定是一个工程细节。很多用户习惯 5 字段 cron，系统建议自动补秒字段，既兼容习惯又保持内部统一。

任务数据会持久化记录 next\_run、最后运行状态、历史执行摘要等信息，为后续审计与恢复提供基础。

### 1.10.2 10.2 Scheduler 与 Agent Loop 的耦合点

Scheduler 并不实现另一套智能体逻辑，而是复用同一个 process\_with\_agent。它只是把任务 prompt 作为 override 注入，让 Agent Loop 以“定时任务来源”身份执行。

这种复用有三个优势：



1. 行为一致：人工触发和定时触发共享同一执行语义。
2. 维护成本低：Bug 修复一次即可覆盖两类入口。
3. 安全策略一致：工具审批、权限、hooks、策略全部复用。

执行完成后，scheduler 会把结果送回对应 chat，并写入运行日志。失败时也会写日志并推送错误消息，让用户知道自动化链路出错而不是沉默失效。

这说明定时系统不是“后台静默脚本”，而是“有反馈的代理执行入口”。

### 1.10.3 10.3 DLQ 与任务恢复机制

自动化系统不可避免会失败。关键不是杜绝失败，而是有可恢复机制。MicroClaw 在任务失败时会写入 DLQ (dead letter queue) 记录，并提供查询和重放工具：

1. `list_scheduled_task_dlq`: 查看失败任务。
2. `replay_scheduled_task_dlq`: 按 chat 或 task 维度重放。

重放策略通常包括：

1. 重新入队并设置立即执行。
2. 在 DLQ 记录标记 `replayed` 状态和注记。
3. 可限制批量重放上限，避免雪崩。

这套机制把“失败任务”从终点变成中间态。配合 runbook 中的 SLO 告警（如 scheduler recoverability），团队可以把自动化可靠性纳入日常运维指标，而非出问题后临时排查。

#### 1.10.4 10.4 本章小结

定时任务系统的核心价值在于：把 Agent 从“被动响应”升级为“主动执行”，同时保留完整的失败记录和恢复路径。

下一章将转向多渠道适配，讨论统一核心如何落地到不同平台约束下的输入输出行为。

## 1.11 第 11 章 多渠道适配层:Telegram、Discord、Web 的共性与差异

### 1.11.1 11.1 渠道路由与消息模型

MicroClaw 的渠道层职责是把外部事件映射为统一内部请求，再把内部响应映射回各平台协议。这个“映射层”是跨平台一致性的关键。

统一请求最少包含：

1. 渠道名称。
2. chat\_id。
3. 会话类型（私聊/群聊等）。
4. 用户消息与发送者信息。

在内部模型中，用户消息会被包装为统一格式（含 sender 信息），并做 XML 转义处理，减少提示注入风险。这个细节说明渠道适配不仅是格式转换，也承担一部分安全清洗职责。

### 1.11.2 11.2 长消息分片与传输约束

不同平台对消息长度和富文本能力限制不同。MicroClaw 在返回层做自动分片，以适配平台上限。分片策略的目标不是“机械切段”，而是尽量在语义边界（如换行）切分，降低可读性损失。

传输层常见差异包括：

1. 文本长度限制差异。
2. 附件上传能力差异。

3. 提及与线程上下文差异。
4. 群聊追赶历史能力差异。

系统把这些差异留在渠道适配器，而不污染 Agent Loop。这样做的长期收益是：新增渠道时，研发重点落在 I/O 适配，不需要重新解释“任务如何执行”。

### **1.11.3 11.3 统一核心与适配器扩展策略**

项目文档明确提出新增平台适配器的关键约束：复用 `process_with_agent`，不要引入平台专属执行循环。这条规则值得强调，因为它直接防止架构腐化。

扩展一个新渠道的推荐步骤：

1. 建立消息入站解析与身份映射。
2. 调用共享 Agent Loop。
3. 把回包策略按平台约束实现。
4. 复用统一存储路径记录消息与会话。
5. 做跨渠道回归，验证行为一致性。

如果某渠道必须有特殊行为，应优先在“适配层”解决，而不是在“核心 loop”加条件分支。否则随着渠道增多，核心会变成不可维护的条件森林。

这种“核心收敛，边缘扩展”的策略在工程上很朴素，但非常有效。它保证 MicroClaw 可以继续扩展渠道而不牺牲核心稳定性。

#### 1.11.4 11.4 本章小结

多渠道支持的关键不是支持多少平台，而是能否在平台差异下保持执行语义一致。MicroClaw 通过适配器隔离输入输出差异、保持共享 Agent Loop，建立了可持续扩展路径。

下一章将讨论 Web 控制面与可观测体系，说明系统如何从“可跑”迈向“可运维”。



## 1.12 第 12 章 Web 控制面与可观测性

### 1.12.1 12.1 Web API 安全面与配置自检

Web 控制面是 MicroClaw 的运维入口，也是高风险入口。与聊天渠道不同，Web 常用于管理动作，安全需求更高。

当前体系包含两部分：

1. 认证授权：逐步从 legacy token 过渡到 session + API key + scope。
2. 配置自检： /api/config/self\_check 输出风险级别和警告项。

自检项覆盖范围很实用：沙箱开关、runtime 可用性、allowlist 文件、web host 暴露、限流阈值、hooks 输出大小、OTLP 配置完整性等。它把“安全建议”转化为“可执行检查”。

这类接口的价值不在于“发现所有问题”，而在于“让高概率错误变得显而易见”。

### 1.12.2 12.2 使用量、记忆与稳定性指标

MicroClaw 的可观测不仅统计请求量，还覆盖工具、调度和记忆注入路径。主要指标面包括：

1. 请求成功率与延迟（端到端）。
2. 工具可靠性与策略阻断率。

3. Scheduler 可恢复率。
4. 记忆注入命中与省略统计。
5. MCP 可靠性（限流、舱壁、断路器拒绝）。

这些指标形成了“Agent 运行健康图谱”。对传统 bot 来说，关注 CPU/RAM 可能足够；对 Agent runtime，必须额外关注“决策-执行链路质量”。

例如工具可靠性下降可能不是服务崩溃，而是审批策略误配；记忆注入下降可能不是 DB 故障，而是 token 预算过低。没有领域指标就看不见这些问题。

### 1.12.3 12.3 运行手册与 SLO 告警闭环

`docs/operations/runbook.md` 展示了一个完整的运维闭环：症状 -> 检查点 -> 快速动作 -> SLO 告警 -> 回滚/修复路径。

关键 SLO 包括：

1. `request_success_rate`。
2. `e2e_latency_p95_ms`。
3. `tool_reliability`。
4. `scheduler_recoverability_7d`。

当 burn alert 触发时，runbook 建议冻结非关键合并、指定 incident owner、必要时准备回滚。这是典型 SRE 实践迁移到 Agent runtime 的做法。

另外, DLQ replay、hooks disable、metrics history 查询等操作都被纳入标准流程, 避免事故处理依赖“谁记得命令”。

可观测体系真正的成熟标志不是图表多, 而是“发生异常时, 团队能在固定时间内完成定位与处置”。从这个标准看, MicroClaw 已经具备清晰的工程化方向。

#### 1.12.4 12.4 本章小结

Web 控制面与可观测体系让 MicroClaw 从“功能可用”走向“运营可控”。自检接口降低配置风险, 指标体系提升诊断效率, runbook 与 SLO 形成事故闭环。

下一章将系统解析配置体系，解释每组参数背后的设计思路与取舍。

## **1.13 第 13 章 配置体系与设计决策：每个参数背后的工程取舍**

### **1.13.1 13.1 配置装载、默认值与兼容策略**

MicroClaw 的配置体系有一个明显特征：默认值丰富且集中定义。这让“可启动”门槛较低，同时为生产化留出显式调优入口。

配置的工程目标可以概括为：

1. 开箱可用：默认值能启动并完成基本流程。

2. 显式可调: 关键风险参数都可覆盖。
3. 向后兼容: 旧字段仍可读取, 迁移可渐进。

例如数据路径默认从当前目录逐步转向 `~/microclaw`, 属于典型“从实验到产品”的迁移方向。默认工作目录和数据目录放在用户目录下, 也降低了跨平台路径差异带来的问题。

兼容策略上, 项目保留了部分 legacy 字段 (如旧渠道字段), 同时鼓励迁移到统一 channels 结构。这样可以避免一次性破坏升级, 又能逐步收敛配置模型。

## 1.13.2 13.2 关键参数组逐项解读

以下按参数组给出实践向解读。

### 1.13.2.1 LLM 与上下文参数

1. `llm_provider / api_key / model / llm_base_url` 这组定义了模型接入边界。`model` 允许为空使用 `provider` 默认值，适合快速启动；生产建议显式固定 `model` 版本，减少行为漂移。
2. `max_tokens` 单次回复上限。过低会截断，过高会增加延迟和成本。推荐按任务形态分层：操作确认类低一些，报告生成类高一些。
3. `max_tool_iterations` Agent Loop 硬上限。默认 100 偏保



守，适合复杂任务；若运行成本敏感，可在 20~60 区间做压测后下调。

4. `max_history_messages`            /  
    `max_session_messages`            /  
    `compact_keep_recent` 这三者共同决定上下文窗口压力。  
    `max_history_messages` 管入站历史，`max_session_messages` 管会话总长，`compact_keep_recent` 管压缩后保留量。
5. `memory_token_budget` 结构化记忆注入预算。过低会“记不住”，过高会“挤占任务上下文”。应结

合 memory injection logs 调优，而非拍脑袋。

### 1.13.2.2 工具与超时参数

1. `default_tool_timeout_secs` 工具超时基础值。默认 30 秒适中，但 `bash`、`browser`、网络工具通常需要单独覆盖。
2. `tool_timeout_overrides` 按工具名覆盖超时预算。推荐优先配置高方差工具，避免长尾阻塞。
3. `default_mcp_request_timeout_secs` MCP 请求超时基线。过短会造成误失败，过长会拖慢整体回合节奏。

### 1.13.2.3 路径与隔离参数

1. `data_dir` 运行时文件和技能目录根路径。生产环境建议固定为持久卷路径, 不随部署目录变化。
2. `working_dir` 工具默认工作目录。建议避免指向系统敏感路径。
3. `working_dir_isolation` `shared` 适合单用户调试; `chat` 适合多会话隔离, 是更安全的默认实践。

### 1.13.2.4 Sandbox 参数

1. `sandbox.mode` `off` 便于上手, `all` 适合风险敏感环境。

2. `sandbox.no_network` 默认 `true`, 减少外联风险。若业务确需联网, 可按最小范围放开并配合审计。
3. `sandbox.require_runtime` 控制缺少 `runtime` 时是回退宿主还是直接失败。生产建议 `true`。
4. `sandbox.mount_allowlist_path` 显式限制可挂载根路径, 推荐在生产强制配置。
5. `sandbox.memory_limit` / `cpu_quota` / `pids_limit` 资源治理参数, 建议按压测数据设定, 不建议留空长期运行。

### 1.13.2.5 Web 参数

1. `web_host` / `web_port` 默认 `loopback` 有助于降低暴露面。若改为公网监听, 应配合反向代理、TLS 和外层鉴权。
2. `web_max_inflight_per_session` 每会话并发上限。过高会放大资源争用。
3. `web_max_requests_per_window` + `web_rate_window_seconds` 限流窗口组合。建议先保守, 基于真实流量逐步提高。
4. `web_session_idle_ttl_seconds` 闲置锁/配额清理窗口。过短会导致频繁抖动, 过长会拖慢回收。

### 1.13.2.6 记忆与反思参数

1. `reflector_enabled` 是否启用后台记忆提取。默认启用，有利于长期任务表现。
2. `reflector_interval_mins` 反思周期。过短会增加成本，过长会降低新信息进入速度。
3. `embedding` 组参数 用于语义检索，需与构建特性匹配 (`sqlite-vec`)。生产中要同时考虑模型成本和向量维度。

### 1.13.2.7 渠道与跨 chat 权限参数

1. `channels.*` 新结构化渠道配置入口。建议只用这一组，减少 `legacy` 字段混用。

2. `control_chat_ids` 跨 chat 操作白名单。该字段是“运维能力开关”，建议最小化配置并定期审计。
3. `bot_username` 与 `channel override` 影响提示词与消息行为，错配会影响提及与识别逻辑。

### 1.13.3 13.3 典型部署配置与风险画像

以下给出三种常见部署画像。

#### 1.13.3.1 画像一：个人开发机

建议配置：

1. `sandbox.mode=off` 或 `all + require_runtime=false`。

2. `working_dir_isolation=chat`。
3. 低并发 Web 配置，保持 `loop-back`。

风险特征：

1. 更偏可用性，边界相对宽松。
2. 需注意不要把工作目录指向含密钥路径。

### 1.13.3.2 画像二：团队测试环境

建议配置：

1. `sandbox.mode=all`，  
`require_runtime=true`。
2. 配置 `mount allowlist`。
3. 打开完整指标与 `runbook` 流程。

风险特征：



1. 目标是提前暴露配置与策略问题。
2. 需要模拟异常（runtime 缺失、审批阻断、超时）做回归。

### 1.13.3.3 画像三：生产自动化环境 建议配置：

1. 强制沙箱 + 资源限制 + allowlist。
2. Web 鉴权使用 scoped API key，禁用 legacy fallback（条件成熟后）。
3. 严格 control\_chat\_ids，并设置告警阈值。

### 风险特征：

1. 关注长期稳定和越权防护。

## 2. 重点监控工具可靠性、调度恢复率、审批拦截率。

配置治理的核心不是“参数都懂”，而是“参数与风险模型绑定”。只有把参数放回具体部署画像，调优才有方向。

### 1.13.4 13.4 本章小结

配置体系是 MicroClaw 工程取舍的外显界面。默认值体现上手策略，覆盖项体现治理能力，兼容层体现演进节奏。

下一章将进入收官，讨论测试、升级、发布和未来路线，完成从代码到系统运营的闭环视角。

## 1.14 第 14 章 从代码到发布： 测试、升级、运维与路线 图

### 1.14.1 14.1 测试基线与稳定性测试

MicroClaw 的测试面不只在函数正确性，还在系统行为一致性。文档与脚本体现了多层测试思路：单元测试、集成测试、稳定性 smoke、权限与恢复性回归。

一个实用测试基线应覆盖：

1. Agent Loop 基础收敛（end\_turn/tool\_use/max\_tokens 分支）。
2. 工具策略校验（risk/policy/approval）。

3. 跨 chat 权限约束。
4. Scheduler 重启恢复与 DLQ replay。
5. Sandbox fallback 与 require\_runtime 行为。
6. Web 限流与并发边界。

这套基线的意义在于防止“功能新增破坏运行时契约”。Agent 系统最怕的不是单点 bug，而是契约漂移：某条边界被悄悄放松，几周后才在生产暴露。

### 1.14.2 14.2 升级路径、回滚策略与发布节奏

从升级指南和近期发布节奏看，MicroClaw 采用“持续小步发布 + 文档同步修复”策略。该策略的优

点是反馈快、风险分散，但前提是升级和回滚路径清晰。

建议升级流程：

1. 先在测试环境应用新版本并跑稳定性 smoke。
2. 检查配置兼容与自检 warning。
3. 观察核心 SLO 一段窗口后再扩大。
4. 保留回滚版本与迁移说明。

回滚策略应提前定义触发条件，例如：

1. 请求成功率持续低于阈值。
2. Scheduler recoverability 显著下降。
3. 权限模型出现越权或误拒绝。

4. 沙箱路径出现与预期不符的执行降级。

版本节奏本身不是目标，稳定交付才是目标。发布频率越高，越需要自动化检查和文档纪律。

### 1.14.3 14.3 技术债与下一阶段演进方向

从 RFC、roadmap 和报告材料看，下一阶段演进可聚焦在以下方向：

1. 安全默认值升级：在更多场景将隔离执行设为默认优先。
2. 授权模型完善：完成 Web scopes 与 legacy 方案切换。
3. Hooks 平台化：稳定事件契约和变更边界。

4. 记忆治理深化：增强 provenance、回滚与质量 SLO。
5. 调度可靠性提升：DLQ 策略和自动补偿更细粒度。
6. 可观测统一：把策略阻断、fall-back、审批流纳入统一报表。

这些方向的共同特点是“强化工程质量而非堆功能”。这是运行时项目走向成熟的典型路径。

对于团队落地而言，最关键的行动建议是：

1. 先建立自己的部署画像和风险阈值。
2. 再把配置、测试、告警全部绑定到画像。

3. 最后才是扩展更多工具和自动化场景。

#### **1.14.4 14.4 本章小结**

本章完成了从开发到运营的闭环视角：功能上线只是起点，稳定运行、可回滚、可演进才是长期价值来源。

至此，全书主体完成。附录将给出术语、工具风险速查、默认配置速查和资料索引，便于读者在实践中快速定位信息。



## 1.15 第 15 章 源码结构化导读：从仓库树到执行路径

### 1.15.1 15.1 仓库分层阅读法

阅读 MicroClaw 代码时，建议先按“执行路径”而不是“目录树”来建立认知。目录树告诉你文件在哪里，执行路径告诉你请求如何流动。推荐阅读顺序如下：

1. 从入口 `src/main.rs` 看启动流程和配置加载。
2. 看 `src/runtime.rs` 识别 `AppState` 的装配关系。
3. 进入 `src/agent_engine.rs` 理解核心执行循环。

4. 看 `src/tools/mod.rs` 理解工具注册、策略校验与分发。
5. 最后看 `src/channels/*`、`src/web/*`、`src/scheduler.rs` 识别外层适配和后台任务。

这种顺序的好处是“先抓主干，再看枝叶”。如果一上来逐文件通读，容易陷入局部细节，最后仍不知道系统是如何连起来的。

### 1.15.2 15.2 crates 级别的职责边界

项目采用工作区多 crate 结构。虽然章节中主要引用 `src/`，但核心逻辑分布在多个 crate：

1. `microclaw-core`：跨模块复用的基础类型与通用工具。

2. microclaw-storage: 数据库封装、迁移、持久化结构。
3. microclaw-tools: 工具运行时策略、沙箱执行器、通用工具协议。
4. microclaw-channels: 渠道抽象与共用逻辑。

该拆分的价值是让“高变动模块”和“稳定模块”分离。比如沙箱与工具策略会频繁调整，核心类型和存储契约更稳定。拆分后，变更影响半径更可控。

### 1.15.3 15.3 src/ 主模块导读

以下按主模块做工程导读。

### **1.15.3.1 src/main.rs**

职责：解析命令行、加载配置、初始化运行时、启动渠道与服务。

关注点：

1. 启动顺序是否可预测。
2. 错误是否在启动期暴露而非运行期才爆炸。
3. 控制命令（如 setup/doctor/hooks）是否和主服务复用同一配置源。

### **1.15.3.2 src/runtime.rs**

职责：构建 AppState, 连接 LLM、数据库、工具注册、记忆、渠道、后台任务。

关注点：

1. 全局依赖是否显式注入。
2. 模块初始化失败时是否有可理解错误。
3. 运行时共享对象是否通过 Arc 做清晰所有权管理。

### **1.15.3.3 src/agent\_engine.rs**

职责：核心 Agent Loop，包括消息准备、提示词构建、工具循环、会话持久化。

关注点：

1. stop\_reason 分支完整性。
2. 空回复与最大迭代边界处理。
3. hooks 与工具执行顺序。
4. 会话恢复与压缩策略是否会损坏协议完整性。

### **1.15.3.4 src/llm.rs**

职责：Provider 适配与请求发送（流式/非流式）。

关注点：

1. 多 provider 能力差异是否被抽象层吸收。
2. 工具调用与文本输出模式是否统一。
3. usage 统计是否可回流到观测层。

### **1.15.3.5 src/tools/mod.rs**

职责：工具注册、定义缓存、执行分发、策略检查、授权注入。

关注点：

1. 策略校验必须在执行前。

2. 工具返回结构是否规范化。
3. 子代理工具集是否严格受限。

### **1.15.3.6 src/scheduler.rs**

职责：定时任务执行、运行日志、DLQ、Reflector。

关注点：

1. 定时任务失败路径是否完整落库。
2. 重放是否具备幂等语义。
3. 反思任务是否引入记忆污染风险。

### **1.15.3.7 src/web/\***

职责：Web API、认证、会话流、配置自检、指标接口。

关注点：

1. scope 校验覆盖是否完整。
2. 限流和并发控制是否可配置。
3. 自检 warning 是否覆盖高频误配。

### **1.15.3.8 src/channels/\***

职责：各平台适配器。

关注点：

1. 进站消息模型是否统一。
2. 出站分片逻辑是否稳定。
3. 平台差异是否泄漏到核心执行层。

### **1.15.4 15.4 工具模块逐项导读**

以下对工具模块做简要代码阅读导图，帮助你在调试时快速定位文件。



1. `bash.rs`: 命令执行入口, 接入 sandbox router 与超时控制。
2. `read_file.rs`: 带行号读取与窗口化输出。
3. `write_file.rs`: 创建/覆盖文件, 路径解析与目录自动创建。
4. `edit_file.rs`: 基于查找替换的可验证编辑。
5. `glob.rs`: 模式匹配搜索。
6. `grep.rs`: 正则内容检索。
7. `web_search.rs`: 网页检索聚合。
8. `web_fetch.rs`: 页面抓取与文本提取。
9. `memory.rs`: 文件记忆读写 + 结构化联动写入。

10. `structured_memory.rs`: 结构化记忆查询/更新/删除。
11. `schedule.rs`: 任务创建、暂停、恢复、取消、历史与 DLQ 操作。
12. `send_message.rs`: 中间消息与附件下发。
13. `export_chat.rs`: 会话导出。
14. `activate_skill.rs`: 技能激活。
15. `sync_skills.rs`: 远端技能同步与本地规范化。
16. `sub_agent.rs`: 受限工具集的并行子代理执行。
17. `browser.rs`: 浏览器自动化入口。
18. `mcp.rs`: MCP 工具桥接。
19. `todo.rs`: 计划状态读写。

调试建议：优先从 `tools/mod.rs` 的注册和策略路径入手，再进入单工具文件。因为很多“工具执行失败”并不是工具实现 bug，而是前置策略或授权阻断。

### 1.15.5 15.5 文档与源码联动阅读法

MicroClaw 的文档目录结构较完整，建议建立“文档到代码”映射：

1. `docs/security/execution-model.md` 对应 `sandbox.rs` 与工具策略校验路径。
2. `docs/operations/runbook.md` 对应 web 指标接口、DLQ 工具与恢复命令。

3. docs/rfcs/\* 对应未来设计方向, 可用于判断当前实现是否临时。
4. docs/generated/\* 对应自动生成的工具/配置清单, 可用于检查文档漂移。

此外, website/blog 的内容通常提供架构视角与里程碑叙事, 适合用于理解“为什么这样设计”, 而 src/\* 用于确认“实际怎么实现”。

### 1.15.6 15.6 阅读与改造的实践清单

当你准备在 MicroClaw 上做二次开发, 可以按以下清单执行:

1. 先画出自己的目标路径: 入口、工具、状态、输出。

2. 确认是否可复用现有 Agent Loop，不新增平行 loop。
3. 新工具先定义风险等级和执行策略，再写实现。
4. 每次策略变更同步补测试。
5. 对外功能变更同步更新 docs/generated。
6. 调整配置默认值时写清迁移影响。
7. 涉及跨 chat 能力时优先检查授权模型。
8. 涉及外部执行能力时优先检查 sandbox 行为。

### 1.15.7 15.7 本章小结

本章给出了 MicroClaw 的结构化读码路径：先执行链，再模块边界，再单文件细节。掌握这种阅读法，

可以显著降低理解成本，也能避免在改造时破坏核心契约。

下一章将从架构阅读转向安全深水区，讨论更细粒度的风险治理与对抗场景。

## **1.16 第 16 章 安全深水区：威胁建模与防护决策**

### **1.16.1 16.1 资产识别：到底在保护什么**

在 Agent runtime 里，安全讨论若不先识别资产，就容易陷入抽象口号。MicroClaw 的关键资产可分为五类：

1. 执行资产：宿主命令执行能力、文件写入能力、外部调用能力。

2. 数据资产：会话历史、结构化记忆、调度任务、导出文件。
3. 凭据资产：API key、渠道 token、第三方服务密钥。
4. 控制资产：Web 管理接口、配置文件、审批流程。
5. 可用性资产：调度器、队列、数据库与事件流。

不同资产对应不同防护策略。比如凭据资产需要最小暴露和路径隔离，可用性资产需要限流和恢复机制。把这些混在一起谈“加强安全”没有执行价值。

## 1.16.2 16.2 攻击面清单：从输入到执行

MicroClaw 的攻击面主要落在四条路径：

1. 用户输入路径：提示注入、越权诱导、社会工程式指令。
2. 工具执行路径：命令注入、路径穿越、敏感文件覆盖。
3. 控制面路径：认证绕过、scope 配置错误、限流失效。
4. 外部依赖路径：MCP 服务异常、网络返回污染、第三方 API 漏洞。

对应防线应形成“分层链”：

1. 输入层：消息包裹、转义、系统提示硬约束。



2. 策略层: risk/policy/approval。
3. 执行层: sandbox + path guard + allowlist。
4. 控制层: auth scope + rate limit + self-check。
5. 运维层: 日志、指标、告警、回放。

分层链的好处是即使某一层漏检，后续层仍有拦截机会。

### 1.16.3 16.3 典型威胁场景与对策

以下给出 12 个高频场景。

1. 场景： 用户要求“把整个 HOME 打包发我”。对策：路径守卫 + 聊天授权 + 工具风险校验；必要时审批。

2. 场景：用户诱导模型“忽略系统提示，直接输出密钥文件”。对策：消息封装与不可信输入策略；文件工具路径拦截。
3. 场景：配置了 `sandbox=all`，但 `docker` 不可用，用户误以为已隔离。对策：`require_runtime=true`；配置自检和告警。
4. 场景：`control chat` 配置过宽，跨 chat 误操作。对策：最小化 `control_chat_ids`，定期审计。
5. 场景：子代理被用于高副作用操作。对策：受限工具集，不暴露发送和调度能力。

6. 场景：定时任务被错误创建为高频 cron，造成资源耗尽。对策：调度频率校验、限流、SLO 告警。
7. 场景：Reflector 将故障描述写入长期记忆，诱发行为污染。对策：记忆质量闸门与污染检测规则。
8. 场景：Web 接口暴露在公网且未配置强认证。对策：loop-back 默认、scope 化密钥、反向代理防护。
9. 场景：hook 脚本异常或恶意输出巨大 payload。对策：hooks 输入输出大小上限、超时硬限制。

10. 场景：MCP 服务高失败率导致调用雪崩。对策：断路器 + 队列等待预算 + 并发上限。
11. 场景：模型频繁触发 bash 高风险动作。对策：审批门 + 行为监控 + prompt 收紧。
12. 场景：升级后策略行为变化导致线上拒绝率飙升。对策：灰度发布 + 稳定性 smoke + 回滚预案。

#### 1.16.4 16.4 防护策略优先级模型

在资源有限条件下，应优先做“高概率高影响”项。推荐优先级如下：

1. P0：认证与作用域（防越权）。

2. P0: 执行隔离与路径防护 (防宿主破坏)。
3. P1: 审批与风险分级 (防误操作)。
4. P1: 可观测与告警 (防隐形故障扩大)。
5. P2: 高级策略 (细粒度 hooks、策略学习等)。

该顺序的原因是：没有基础边界，复杂策略只会增加误判成本。

### **1.16.5 16.5 安全评审模板**

建议每次版本迭代执行一次简化安全评审，模板如下：

1. 本版本新增了哪些执行能力？
2. 是否新增了跨 chat / 跨系统权限路径？

3. 对应工具风险等级是否更新？
4. 沙箱策略是否被弱化？
5. 自检告警是否新增高危项？
6. 监控面是否覆盖新增路径？
7. 回滚路径是否可执行且已验证？

每个问题要求给出证据：代码位置、测试结果、文档变更。

### 1.16.6 16.6 安全运营建议

长期运行建议：

1. 每周审计 `control_chat_ids` 与 `API key scopes`。
2. 每周检查 `sandbox_runtime_unavailable` 类告警。
3. 每日抽样检查高风险工具调用日志。

4. 每次发布后 24 小时重点观察策略阻断率。
5. 每月演练一次 DLQ replay 与回滚流程。

安全不应是发布前一次性动作，而是运行中的持续治理。

### 1.16.7 16.7 本章小结

本章将安全机制从“代码能力”扩展到“威胁场景 + 治理流程”。MicroClaw 的安全价值不在于某个单点特性，而在于多层防线与持续运营结合。

下一章将把 Sandbox 拉到实战层，用具体操作与攻防案例讨论隔离机制的边界与改进空间。

## 1.17 第 17 章 Sandbox 实战 与攻防：部署、诊断与演 练

### 1.17.1 17.1 三种环境的沙箱模板

#### 1.17.1.1 模板 A：开发机（快速迭代）

- 目标：降低摩擦，保证开发效率。
- 建议：

1. `sandbox.mode=off` 或 `all + require_runtime=false`。
2. 保持 `working_dir_isolation=chat`。
3. 限制工作目录到项目路径。



### 1.17.1.2 模板 B：测试环境（策略验证）

- 目标：提前验证隔离策略和兼容性。
- 建议：
  1. `sandbox.mode=all`。
  2. `require_runtime=true`。
  3. 配置 `mount allowlist`。
  4. 开启资源限制并记录执行失败类型。

### 1.17.1.3 模板 C：生产环境（边界优先）

- 目标：稳定、可审计、可回滚。
- 建议：
  1. 强制沙箱 + `fail-closed`。

2. 明确 allowlist 与敏感目录策略。
3. 为高风险工具建立审批和告警。
4. 周期检查 runtime 可用性。

### 1.17.2 17.2 上线前验证清单

1. 运行 doctor sandbox, 确认 runtime 可用。
2. 检查 sandbox.mode、require\_runtime 与部署画像一致。
3. 验证 allowlist 文件存在且有条目。
4. 验证敏感路径被阻断（如 .ssh 相关路径）。
5. 验证符号链接路径被拒绝。
6. 验证超时行为不会卡住主流程。
7. 验证资源限制参数生效。

8. 验证 fallback/blocked 事件会进入日志与监控。

这份清单建议纳入 CI 或预发布脚本，而不是人工口头检查。

### **1.17.3 17.3 典型故障与排障流程**

#### **1.17.3.1 故障 1：沙箱已开启但仍在宿主执行**

排障步骤：

1. 查配置是否 mode=all。
2. 查 runtime 是否可用。
3. 查 require\_runtime 是否 false 导致 fallback。
4. 查日志中的 fallback warning。
5. 查 self-check 的风险警告输出。

### **1.17.3.2 故障 2：容器执行频繁超时**

排障步骤：

1. 检查默认超时是否过低。
2. 检查命令本身是否需要拆分。
3. 检查资源限制是否过紧。
4. 检查工作目录挂载是否可访问。

### **1.17.3.3 故障 3：挂载路径被拒绝**

排障步骤：

1. 检查路径是否包含符号链接。
2. 检查路径组件是否命中敏感名单。
3. 检查 allowlist 是否覆盖该根路径。
4. 在测试环境复现并记录具体拒绝原因。

#### 1.17.4 17.4 攻防演练案例 (10 例)

1. 演练：尝试访问 `~/.ssh` 目录。  
预期：路径校验拒绝，工具返回错误并记录。
2. 演练：在无 `docker` 环境启用 `mode=all`。预期：  
`require_runtime=true` 时直接失败，`false` 时告警回退。
3. 演练：通过软链接绕过 `allowlist`。预期：符号链接检查拦截。
4. 演练：高频长命令导致资源压测。预期：受 CPU/内存/pids 限制约束，超时可控。

5. 演练：用户诱导执行破坏性命令。预期：审批门与策略日志可见。
6. 演练：在 control chat 触发跨 chat 写文件。预期：权限通过但仍受路径和策略约束。
7. 演练：在普通 chat 触发跨 chat 操作。预期：授权拒绝。
8. 演练：模拟 docker 临时不可用。预期：行为与 require\_runtime 配置一致。
9. 演练：挂载目录包含 .env.production 文件片段。预期：敏感组件规则触发告警/拒绝。

10. 演练：命令执行过程中进程失控。预期：超时终止并返回结构化错误。

### 1.17.5 17.5 沙箱策略演进建议

1. 逐步扩大 sandbox-only 工具覆盖范围（先高风险）。
2. 把 fallback 行为纳入强告警与发布阻断规则。
3. 为不同环境预制策略矩阵（desktop/server/CI）。
4. 结合工作负载做资源上限分层模板。
5. 对敏感目录名单与 allowlist 机制做团队级标准化。

### **1.17.6 17.6 本章小结**

Sandbox 的核心不在“是否使用 Docker”，而在“隔离策略是否被持续执行并可审计”。本章给出了从部署到演练的落地路径。

下一章将回到 Agent Loop，以案例工坊方式拆解复杂任务如何在多轮中稳定收敛。

## **1.18 第 18 章 Agent Loop 案例工坊：30 个任务如何收敛**

### **1.18.1 18.1 使用说明**

本章给出 30 个典型任务案例。每个案例都按同一模板组织：

1. 用户目标。



2. 建议循环路径。
3. 关键工具调用。
4. 常见失败点与修复策略。

这组案例的目的不是提供固定答案，而是提供“可复用执行骨架”。

## 1.18.2 18.2 案例集

### 1.18.2.1 案例 1：仓库超时配置普查

- 目标：找出所有 timeout 配置并汇总。
- 路径：todo -> glob -> grep -> read\_file -> 汇总。
- 失败点：grep 结果过大。
- 修复：先按目录分批检索，逐批汇总。

### 1.18.2.2 案例 2:生成发布说明草稿

- 目标：根据最近提交生成 release notes。
- 路径：bash(git log) -> 分类 -> 输出。
- 失败点：提交跨度过大。
- 修复：限定时间窗口或版本 tag。

### 1.18.2.3 案例 3：定位某错误日志根因

- 目标：从日志中提取错误链路。
- 路径：read\_file -> grep(error) -> grep(trace) -> 汇总。
- 失败点：日志滚动导致缺失。
- 修复：多文件合并视图，优先最近窗口。

#### 1.18.2.4 案例4:批量修改配置键名

- 目标：把旧配置键迁移到新键。
- 路径：glob -> read\_file -> edit\_file -> 验证。
- 失败点：替换误伤注释。
- 修复：限制替换上下文并二次 grep 校验。

#### 1.18.2.5 案例5:多文档摘要为操作清单

- 目标：把分散文档转成执行列表。
- 路径：read\_file(多文档) -> todo\_write -> 逐项输出。
- 失败点：摘要太泛。
- 修复：每项要求“动作 + 负责人 + 条件”。

### 1.18.2.6 案例 6：周期性情报播报

- 目标：每天固定时间汇总新闻。
- 路径：schedule\_task(cron) -> web\_search -> web\_fetch -> send\_message。
- 失败点：源站不稳定。
- 修复：配置多源兜底与失败摘要。

### 1.18.2.7 案例 7：跨 chat 广播（控制会话）

- 目标：在多个 chat 发布通知。
- 路径：权限检查 -> send\_message(target\_ch
- 失败点：非 control chat 调用。
- 修复：回退当前 chat 并提示权限不足。

### 1.18.2.8 案例 8：记住用户偏好并回读

- 目标：保存并验证偏好。
- 路 径： write\_memory -> read\_memory -> 回复确认。
- 失败点：信息过于模糊。
- 修复：要求用户提供可操作描述。

### 1.18.2.9 案例 9：从长对话提取长期事实

- 目标：避免信息遗忘。
- 路径： reflector 定时提取 -> structured memory 注入。
- 失败点：提取噪声过多。
- 修复：提升质量闸门与分类规则。

### 1.18.2.10 案例 10: 子代理并行调研 依赖风险

- 目标: 分析依赖库安全与维护状态。
- 路径: `sub_agent(task)` -> 主代理整合。
- 失败点: 子代理任务定义不清。
- 修复: 加入输入边界和输出格式约束。

### 1.18.2.11 案例 11: 检查 Web 配置 风险

- 目标: 判断当前部署是否安全。
- 路径: 调用 `self_check` -> 解析 `warning` -> 给整改清单。
- 失败点: `warning` 信息太多。
- 修复: 按高/中/低优先级输出。

### 1.18.2.12 案例 12: 恢复失败的定时任务

- 目标: 处理 DLQ 中断任务。
- 路径: list\_scheduled\_task\_dlq -> replay -> 观察结果。
- 失败点: 重放后再次失败。
- 修复: 先修配置再重放, 避免死循环。

### 1.18.2.13 案例 13: 自动化代码体检

- 目标: 每日检查仓库关键指标。
- 路径: schedule -> bash(测试/静态检查) -> 输出简报。
- 失败点: 命令波动大。
- 修复: 拆成多个小任务并独立重试。

### 1.18.2.14 案例 14: 定位权限拒绝原因

- 目标: 解释 Permission denied。
- 路径: 检查 auth context -> control\_chat\_ids -> target\_chat。
- 失败点: 用户以为系统故障。
- 修复: 输出清晰权限说明与申请路径。

### 1.18.2.15 案例 15: 知识库链接失效修复

- 目标: 扫描并修复文档坏链。
- 路径: glob(md) -> grep(http) -> web\_fetch 验证。
- 失败点: 网络抖动误判。



- 修复：二次验证 + 缓存失败样本。

#### **1.18.2.16 案例 16：大型任务拆解失败**

- 目标：任务太大，循环触顶。
- 路径：检测 max iterations -> 提示拆分。
- 失败点：用户继续大请求。
- 修复：自动提供拆分模板。

#### **1.18.2.17 案例 17：工具连续失败后的用户沟通**

- 目标：避免误导“已完成”。
- 路径：收集 failed\_tools -> 透明反馈。
- 失败点：失败信息过于技术化。

- 修复：分“用户可执行动作”和“内部原因”两段输出。

#### **1.18.2.18 案例 18: 群聊上下文噪声过高**

- 目标：提取与任务有关信息。
- 路径：消息窗口限制 -> 相关关键词过滤。
- 失败点：遗漏关键前文。
- 修复：允许用户指定“从某条消息开始”。

#### **1.18.2.19 案例 19: 跨平台消息分片不一致**

- 目标：保持不同平台可读性。
- 路径：统一语义输出 -> 适配器分片。
- 失败点：段落断裂。

- 修复：按换行与标题边界切分。

### 1.18.2.20 案例 20：工具返回结构异常

- 目标：防止系统崩溃。
- 路径：ToolResult 规范化 -> 缺省字段补全。
- 失败点：error\_type 缺失。
- 修复：运行时自动标注 tool\_error。

### 1.18.2.21 案例 21：沙箱 runtime 波动

- 目标：确认执行路径是否可靠。
- 路径：观测 fallback 计数 -> 策略调整。
- 失败点：误以为一直在沙箱。

- 修复：开启 `require_runtime` 并阻断发布。

#### **1.18.2.22 案例 22：记忆冲突更新**

- 目标：用户偏好发生变化。
- 路径：显式记忆命令 -> `super-sede`。
- 失败点：旧记忆仍被注入。
- 修复：清理归档状态和优先级。

#### **1.18.2.23 案例 23：技能使用时机错误**

- 目标：避免无关技能污染流程。
- 路径：先 `todo` 再 `activate_skill`，再执行。
- 失败点：技能滥用。
- 修复：在任务定义中明确触发条件。

### 1.18.2.24 案例 24：长文本输出中断

- 目标：完整输出报告。
- 路径：检测 max\_tokens -> 提示继续。
- 失败点：用户误以为已结束。
- 修复：自动附“继续输出”建议语。

### 1.18.2.25 案例 25：外部网页抓取超时

- 目标：快速给出阶段结果。
- 路径：web\_fetch 超时 -> 提示部分结果。
- 失败点：全量失败导致无输出。
- 修复：先返回已抓取部分并标记缺口。

### 1.18.2.26 案例 26：模型返回空可见文本

- 目标：保证用户看到结果。
- 路径：runtime guard 重试一次。
- 失败点：重试后仍空。
- 修复：返回明确失败提示并建议重试。

### 1.18.2.27 案例 27：文件编辑竞态

- 目标：避免覆盖最新改动。
- 路径：read -> edit -> read 回检。
- 失败点：并发写入。
- 修复：增加唯一匹配约束与变更确认。

### 1.18.2.28 案例 28：高频任务导致告警风暴

- 目标：控制告警噪声。
- 路径：指标聚合 -> 阈值分级 -> 抑制策略。
- 失败点：每次失败都告警。
- 修复：按窗口聚合并设置恢复条件。

### 1.18.2.29 案例 29：升级后行为漂移

- 目标：快速验证核心契约。
- 路径：稳定性 smoke -> 比对关键指标。
- 失败点：只看功能是否能跑。
- 修复：把策略拒绝率和恢复率纳入验收。

### 1.18.2.30 案例 30: 用户要求“全自动无确认执行”

- 目标: 平衡效率与风险。
- 路径: 保留审批门 + 提供批处理方案。
- 失败点: 越权自动化。
- 修复: 明确不能绕过安全边界。

### 1.18.3 18.3 模板化执行建议

对于多数复杂任务, 你可以直接套用以下模板:

1. 用 `todo_write` 明确阶段目标。
2. 每阶段只选择必要工具。
3. 每阶段结束更新 `todo` 状态。
4. 阶段失败立即暴露, 不隐瞒。
5. 最终总结包含“完成项 + 失败项 + 下一步”。



#### **1.18.4 18.4 本章小结**

案例工坊的核心不是“记住 30 个答案”，而是形成统一执行肌肉记忆：先计划、再执行、可见失败、可持续推进。

下一章将进入配置百科，把参数设计和故障模式逐项打通。

### **1.19 第 19 章 配置项设计百科：意图、风险与反模式**

#### **1.19.1 19.1 阅读方式**

本章按“配置项字典”形式展开。每个条目包含四个维度：

1. 设计意图：这个参数为什么存在。

2. 风险点：配置不当会出什么问题。
3. 推荐策略：常见环境下如何设置。
4. 反模式：最常见误配方式。

## 1.19.2 19.2 LLM 组

### 1.19.2.1 1) llm\_provider

- 设计意图：抽象多家模型提供商。
- 风险点：provider 能力差异导致行为漂移。
- 推荐策略：生产固定 provider 并版本化。
- 反模式：在同环境频繁切换 provider 但无回归。

### 1.19.2.2 2) api\_key

- 设计意图：统一 provider 凭据注入。
- 风险点：空值在某些 provider 下会启动失败。
- 推荐策略：通过环境注入并避免落盘明文。
- 反模式：把 key 直接提交到配置文件仓库。

### 1.19.2.3 3) model

- 设计意图：显式模型选择。
- 风险点：默认模型变化造成不可控升级。
- 推荐策略：生产固定模型名，升级走灰度。

- 反模式：依赖 provider 默认模型且无监控。

#### **1.19.2.4 4) llm\_base\_url**

- 设计意图：支持私有网关与兼容端点。
- 风险点：网关不兼容工具调用协议。
- 推荐策略：上线前做工具回合兼容压测。
- 反模式：替换网关后只测纯文本对话。

#### **1.19.2.5 5) max\_tokens**

- 设计意图：限制单次输出长度。
- 风险点：过低导致关键信息截断。

- 推荐策略：按任务模板分层配置。
- 反模式：为了省钱统一设置很小上限。

#### **1.19.2.6 6) max\_tool\_iterations**

- 设计意图：限制循环失控。
- 风险点：过低无法完成复杂任务；过高耗时增加。
- 推荐策略：先保守高值，再用数据下调。
- 反模式：为了快把上限压到个位数。

### **1.19.3 19.3 上下文与记忆组**

#### **1.19.3.1 7) max\_history\_messages**

- 设计意图：控制历史窗口规模。

- 风险点：窗口过大导致噪声堆积。
- 推荐策略：群聊比私聊更保守。
- 反模式：把它当“越大越聪明”指标。

#### **1.19.3.2 8) max\_session\_messages**

- 设计意图：触发压缩前的会话上限。
- 风险点：过高导致会话臃肿。
- 推荐策略：结合 compaction 观察延迟。
- 反模式：只调它，不调压缩保留量。

#### **1.19.3.3 9) compact\_keep\_recent**

- 设计意图：压缩后保留近期上下文。

- 风险点：过低丢失最近关键动作。
- 推荐策略：保持足够覆盖最近执行轨迹。
- 反模式：为了节省 token 把它设为极小。

#### **1.19.3.4 10) memory\_token\_budget**

- 设计意图：限制记忆注入成本。
- 风险点：预算过低导致“假失忆”。
- 推荐策略：配合注入日志迭代调优。
- 反模式：只看主观体验，不看注入统计。

### **1.19.3.5 11) reflector\_enabled**

- 设计意图：控制后台记忆提取开关。
- 风险点：关闭后长期记忆演进停滞。
- 推荐策略：生产默认开启。
- 反模式：因一次误提取就永久关闭。

### **1.19.3.6 12)**

#### **reflector\_interval\_mins**

- 设计意图：平衡提取频率与成本。
- 风险点：间隔过短会增加系统负担。
- 推荐策略：按对话密度分环境设置。



- 反模式：统一高频而不看业务活跃度。

## 1.19.4 19.4 路径与隔离组

### 1.19.4.1 13) data\_dir

- 设计意图：统一运行时数据根目录。
- 风险点：路径变化导致状态迁移混乱。
- 推荐策略：生产绑定稳定持久卷。
- 反模式：部署脚本每次生成新路径。

### 1.19.4.2 14) skills\_dir

- 设计意图：可选覆盖技能目录。

- 风险点：与 data\_dir 技能目录混用产生歧义。
- 推荐策略：团队内统一一种来源。
- 反模式：同环境多技能目录并存。

#### **1.19.4.3 15) working\_dir**

- 设计意图：工具默认工作路径。
- 风险点：误指向系统敏感目录。
- 推荐策略：专用目录 + 权限最小化。
- 反模式：直接指向用户 home 根目录。

#### **1.19.4.4 16)**

##### **working\_dir\_isolation**

- 设计意图：共享或按 chat 隔离。

- 风险点：shared 模式跨会话污染。
- 推荐策略：多用户环境优先 chat。
- 反模式：为了方便全量使用 shared。

#### **1.19.4.5 17) timezone**

- 设计意图：调度时区统一。
- 风险点：时区错配导致任务错时执行。
- 推荐策略：显式配置并与团队约定一致。
- 反模式：依赖系统默认时区。

## 1.19.5 19.5 Sandbox 组

### 1.19.5.1 18) `sandbox.mode`

- 设计意图：控制执行隔离开关。
- 风险点：off 模式在高风险场景暴露宿主。
- 推荐策略：生产逐步迁移到 all。
- 反模式：认为开启一次后永远安全。

### 1.19.5.2 19) `sandbox.backend`

- 设计意图：后端选择（auto/docker）。
- 风险点：auto 下环境差异造成行为不一致。
- 推荐策略：关键环境固定后端并验收。

- 反模式：跨环境混用且不做自检。

#### **1.19.5.3 20) `sandbox.image`**

- 设计意图：指定容器执行镜像。
- 风险点：镜像依赖导致命令失败。
- 推荐策略：维护组织级基线镜像。
- 反模式：临时换镜像但不做回归。

#### **1.19.5.4 21) `sandbox.container_prefix`**

- 设计意图：容器命名隔离。
- 风险点：前缀冲突导致清理困难。
- 推荐策略：按环境设置唯一前缀。

- 反模式：多实例共用同前缀。

#### **1.19.5.5 22) `sandbox.no_network`**

- 设计意图：限制容器外联。
- 风险点：误关闭导致数据外传面扩大。
- 推荐策略：默认 true，按需放开。
- 反模式：为图省事全局关闭网络隔离。

#### **1.19.5.6 23)**

##### **`sandbox.require_runtime`**

- 设计意图：runtime 不可用时行为策略。
- 风险点：false 导致静默回退宿主。
- 推荐策略：生产设为 true。
- 反模式：不知道其值就上线。

#### **1.19.5.7 24) `sandbox.mount_allowlist_path`**

- 设计意图：显式挂载白名单。
- 风险点：缺失时挂载边界过宽。
- 推荐策略：配置并版本化管理。
- 反模式：上线后从不维护 allowlist。

#### **1.19.5.8 25)**

##### **`sandbox.memory_limit`**

- 设计意图：限制内存占用。
- 风险点：过小导致频繁 OOM。
- 推荐策略：按压测结果分层配置。
- 反模式：拍脑袋设置固定小值。

#### **1.19.5.9 26) `sandbox.cpu_quota`**

- 设计意图：控制 CPU 份额。

- 风险点：过低导致执行超时增多。
- 推荐策略：按任务类型设置范围。
- 反模式：所有环境统一极限压缩。

#### **1.19.5.10 27) sandbox.pids\_limit**

- 设计意图：限制进程数。
- 风险点：某些工具链需要更多子进程。
- 推荐策略：先测后配，留安全余量。
- 反模式：不测试就设置过低。



## 1.19.6 19.6 Web 控制面组

### 1.19.6.1 28) web\_enabled

- 设计意图：控制 Web 控制面开关。
- 风险点：无需求暴露额外攻击面。
- 推荐策略：不用就关闭。
- 反模式：默认开启却从不维护。

### 1.19.6.2 29) web\_host

- 设计意图：绑定监听地址。
- 风险点：0.0.0.0 暴露公网风险。
- 推荐策略：默认 loopback, 公网需额外防护。
- 反模式：开发便捷配置直接复制到生产。

### **1.19.6.3 30) web\_port**

- 设计意图：控制入口端口。
- 风险点：与现有服务冲突。
- 推荐策略：纳入基础设施端口管理。
- 反模式：多实例共享同端口。

### **1.19.6.4 31) web\_auth\_token**

- 设计意图：legacy token 鉴权。
- 风险点：单 token 粒度粗、轮换困难。
- 推荐策略：迁移到 scoped API key。
- 反模式：长期依赖单 token。

### **1.19.6.5 32) web\_max\_inflight\_per\_session**

- 设计意图：控制会话并发。
- 风险点：过高导致资源争抢。

- 推荐策略：先低后高，逐步放量。
- 反模式：按峰值一次性放开。

#### **1.19.6.6 33) web\_max\_requests\_per\_window**

- 设计意图：窗口内请求限额。
- 风险点：过大导致突发洪峰。
- 推荐策略：结合窗口长度共同调参。
- 反模式：只调请求数不调窗口。

#### **1.19.6.7 34)**

##### **web\_rate\_window\_seconds**

- 设计意图：限流时间窗口。
- 风险点：过短但额度高，等效放水。
- 推荐策略：与请求额度成比例设置。

- 反模式：为了“快”把窗口压太短。

#### 1.19.6.8 35)

##### **web\_run\_history\_limit**

- 设计意图：流式历史缓存上限。
- 风险点：过小影响回放，过大占内存。
- 推荐策略：按会话并发估算。
- 反模式：忽略历史面板需求。

#### 1.19.6.9 36) web\_session\_idle\_ttl\_seconds

- 设计意图：闲置会话清理。
- 风险点：过低引发频繁锁抖动。
- 推荐策略：不少于用户常见停顿时长。
- 反模式：设得极短导致体验波动。

## 1.19.7 19.7 渠道与权限组

### 1.19.7.1 37) `control_chat_ids`

- 设计意图：定义跨 chat 操作特权。
- 风险点：配置过宽会放大误操作半径。
- 推荐策略：最小化，定期审计。
- 反模式：把所有 chat 都设成 control。

### 1.19.7.2 38) `channels.telegram.enabled`

- 设计意图：控制 Telegram 适配器开关。
- 风险点：误开导致无 token 启动异常。
- 推荐策略：按部署目标启用。

- 反模式：模板复制后忘记关闭未用渠道。

#### **1.19.7.3 39) `channels.telegram.bot_token`**

- 设计意图：Telegram 凭据。
- 风险点：泄露后可被接管。
- 推荐策略：密钥托管 + 定期轮换。
- 反模式：明文写入共享文档。

#### **1.19.7.4 40) `channels.telegram.allowed_groups`**

- 设计意图：群组白名单。
- 风险点：空列表即全开放。
- 推荐策略：生产用白名单精确控制。
- 反模式：不知道空列表语义。

#### **1.19.7.5 41) `channels.discord.enabled`**

- 设计意图：控制 Discord 适配器。
- 风险点：误启造成多渠道意外响应。
- 推荐策略：按环境显式启停。
- 反模式：所有渠道默认全开。

#### **1.19.7.6 42) `channels.discord.allowed_channel`**

- 设计意图：限制可响应频道。
- 风险点：未设会扩大响应范围。
- 推荐策略：仅开放业务频道。
- 反模式：测试期配置长期保留。

#### **1.19.7.7 43)**

##### **`channels.web.enabled`**

- 设计意图：统一 Web 频道启用位。

- 风险点：与 legacy web\_enabled 混淆。
- 推荐策略：统一使用新字段。
- 反模式：新旧字段同时改，结果不可预期。

## **1.19.8 19.8 语音、技能与扩展组**

### **1.19.8.1 44) voice\_provider**

- 设计意图：语音转写 provider 选择。
- 风险点：本地命令路径不可用。
- 推荐策略：先验证命令模板。
- 反模式：切换 provider 不做回归。

### **1.19.8.2 45) voice\_transcription\_command**

- 设计意图：本地转写命令模板。



- 风险点：命令注入与依赖缺失。
- 推荐策略：固定模板并限制参数来源。
- 反模式：允许用户输入直接拼接命令。

#### **1.19.8.3 46) clawhub\_registry**

- 设计意图：技能仓库地址。
- 风险点：不可信源带来供应链风险。
- 推荐策略：固定可信 registry。
- 反模式：临时切换未知源。

#### **1.19.8.4 47) clawhub\_token**

- 设计意图：访问私有技能仓库。
- 风险点：token 泄露可导致越权下载。
- 推荐策略：最小权限 token。

- 反模式：高权限 token 长期不轮换。

#### **1.19.8.5 48) clawhub\_agent\_tools\_enabled**

- 设计意图：是否开放 agent 侧 clawhub 工具。
- 风险点：开放后技能安装面扩大。
- 推荐策略：按治理能力决定是否开启。
- 反模式：开启后无审计。

### **1.19.9 19.9 费用与可观测组**

#### **1.19.9.1 49) model\_prices**

- 设计意图：成本估算映射表。
- 风险点：价格未更新导致成本误判。

- 推荐策略：定期同步真实价格。
- 反模式：把历史价格长期当真。

### **1.19.9.2 50) openai\_api\_key (转写等扩展场景)**

- 设计意图：隔离语音能力凭据。
- 风险点：与主模型 key 混用造成权限不清。
- 推荐策略：按功能拆分 key。
- 反模式：一个 key 覆盖所有服务。

### **1.19.10 19.10 配置治理总原则**

1. 把配置看作“系统行为编码”，不是部署杂项。
2. 每次改配置都要有变更理由和回滚路径。

3. 安全参数默认收敛, 效率参数逐步放开。
4. 参数调优要有观测数据支撑。
5. 将高风险配置纳入自动自检与发布门禁。

### 1.19.11 19.11 本章小结

本章将配置项从“字段列表”转化为“行为与风险映射”。理解配置的设计意图, 才能在不同部署环境中做出稳健决策。

下一章将以事故复盘形式, 展示如何用这些机制处理真实故障。

## 1.20 第 20 章 事故复盘与演练：从告警到恢复

### 1.20.1 20.1 复盘方法

本章采用统一复盘模板：

1. 事故现象。
2. 影响范围。
3. 根因分析。
4. 处置步骤。
5. 长期改进。

通过统一模板，团队可以把“个人经验”转成“组织知识”。

## 1.20.2 20.2 事故案例（20 例）

### 1.20.2.1 事故 1：请求成功率突降

- 现象：  
slo.request\_success\_rate 连续低于阈值。
- 影响：多渠道请求失败增多。
- 根因：某次发布后工具策略误配，关键工具被阻断。
- 处置：回滚策略映射，重跑 smoke。
- 改进：把策略映射校验加入 CI。

### 1.20.2.2 事故 2：工具可靠性下降

- 现象：tool\_reliability 下滑。
- 影响：任务完成率下降。
- 根因：外部命令超时预算过低。
- 处置：调高高方差工具超时。

- 改进：建立按工具分类的超时基线。

### 1.20.2.3 事故 3：scheduler 恢复率异常

- 现象：  
scheduler\_recoverability\_7d 下降。
- 影响：定时任务漏执行。
- 根因：重启后部分任务状态未正确更新。
- 处置：手动 replay DLQ，修复状态迁移。
- 改进：增加重启恢复回归用例。

### 1.20.2.4 事故 4：沙箱误回退宿主

- 现象：日志出现大量 fallback。
- 影响：隔离边界弱化。

- 根因：docker runtime 不可用且 `require_runtime=false`。
- 处置：切换 fail-closed 并修复 runtime。
- 改进：发布门禁加入 runtime 可用性检测。

#### **1.20.2.5 事故 5：跨 chat 误发消息**

- 现象：消息发送到错误 chat。
- 影响：隐私与业务风险。
- 根因：control chat 白名单配置过宽。
- 处置：收缩白名单并补审计日志。
- 改进：control chat 变更双人审核。



### 1.20.2.6 事故 6: Web 控制面暴露

- 现象：外网可直接访问管理接口。
- 影响：潜在越权风险。
- 根因：web\_host 配置为公网地址且防护缺失。
- 处置：回退到 loopback, 外层代理加鉴权。
- 改进：自检高危项触发发布阻断。

### 1.20.2.7 事故 7: 记忆污染导致行为异常

- 现象：模型反复复述错误行为。
- 影响：回答质量持续退化。
- 根因：反思写入了“故障复述”类记忆。

- 处置：清理污染记忆，补过滤规则。
- 改进：记忆质量审计仪表板上线。

#### **1.20.2.8 事故 8：模型空可见回复增多**

- 现象：用户看到空回复。
- 影响：交互失败。
- 根因：provider 输出结构变化。
- 处置：启用 guard 重试并加兼容处理。
- 改进：增加 provider 兼容回归测试。

#### **1.20.2.9 事故 9：高频任务导致资源拥塞**

- 现象：CPU 飙高，排队严重。

- 影响：系统整体延迟上升。
- 根因：多个 cron 任务配置过密。
- 处置：限流、降频、分批执行。
- 改进：调度配置增加频率守卫。

#### 1.20.2.10 事故 10：hooks 输出过大

- 现象：hook 处理链路异常缓慢。
- 影响：请求时延增加。
- 根因：hook 返回过大 JSON。
- 处置：降低输出上限并压缩字段。
- 改进：为 hook 增加体积监控。

#### 1.20.2.11 事故 11：升级后配置字段歧义

- 现象：新旧字段冲突导致行为不可预测。

- 影响：渠道启停异常。
- 根因：legacy 与新 channels 混用。
- 处置：统一迁移到新字段。
- 改进：升级脚本输出冲突检测报告。

#### **1.20.2.12 事故 12：MCP 调用雪崩**

- 现象：外部服务慢导致主流程堆积。
- 影响：请求超时显著增多。
- 根因：并发与队列上限不足。
- 处置：启用 bulkhead 与断路器参数。
- 改进：建立每服务容量模型。

### 1.20.2.13 事故 13：日志不足难以追责

- 现象：事故后无法还原执行链。
- 影响：修复周期拉长。
- 根因：关键事件未结构化记录。
- 处置：补 AgentEvent 与工具日志字段。
- 改进：统一日志 schema。

### 1.20.2.14 事故 14：长任务反复触顶迭代上限

- 现象：多轮均提示 max iterations。
- 影响：任务无法收敛。
- 根因：缺少计划分解与中间收敛点。
- 处置：强制 todo 拆分再执行。

- 改进：复杂任务自动建议拆分模板。

### **1.20.2.15 事故 15：导出功能权限误配**

- 现象：普通 chat 可导出非本 chat 内容。
- 影响：数据泄露风险。
- 根因：某版本授权校验回归。
- 处置：紧急 hotfix + 回归测试。
- 改进：权限矩阵测试纳入必跑。

### **1.20.2.16 事故 16：技能同步污染本地环境**

- 现象：同步后技能行为异常。
- 影响：任务执行稳定性下降。
- 根因：来源不可信或 frontmatter 不规范。

- 处置：回滚技能目录，重新同步可信源。
- 改进：技能来源白名单与签名策略。

### **1.20.2.17 事故 17：数据库文件空间膨胀**

- 现象：磁盘占用持续上升。
- 影响：I/O 性能下降。
- 根因：历史记录与日志未治理。
- 处置：归档与清理策略落地。
- 改进：容量告警与周期压缩计划。

### **1.20.2.18 事故 18：不同渠道行为不一致**

- 现象：同请求在不同渠道结果差异明显。

- 影响：用户体验割裂。
- 根因：适配层绕过共享执行路径。
- 处置：回收分叉逻辑，统一走核心 loop。
- 改进：跨渠道一致性回归套件。

### **1.20.2.19 事故 19：审批流程被误解为失败**

- 现象：用户把 `approval_required` 当错误。
- 影响：任务中断率上升。
- 根因：交互反馈不清晰。
- 处置：改进审批提示语与自动重试说明。
- 改进：审批事件可视化。



### 1.20.2.20 事故 20：告警太多导致 疲劳

- 现象：团队忽视真实高危告警。
- 影响：故障发现延迟。
- 根因：阈值与分级设计不合理。
- 处置：重构告警分层与抑制规则。
- 改进：按业务影响重排告警优先级。

### 1.20.3 20.3 演练计划（季度版）

建议每季度至少执行一次演练组合：

1. 权限越权演练。
2. 沙箱 runtime 不可用演练。
3. 调度恢复与 DLQ 重放演练。
4. Web 鉴权与限流演练。

## 5. 记忆污染清理演练。

每次演练都应输出：时间线、证据、改进项、负责人、截止日期。

### 1.20.4 20.4 复盘文化建议

1. 复盘聚焦系统，不追责个人。
2. 结论必须可执行，可验证。
3. 每个改进项必须绑定 owner。
4. 未关闭改进项必须进入下一轮复盘。
5. 复盘文档应纳入知识库检索。

### 1.20.5 20.5 本章小结

事故复盘是运行时系统成熟的关键能力。它把“出问题再修”升级为“可学习、可复用、可预防”的工程循环。

至此，扩展卷完成。全书最后将给出总总结与实践路线建议。

## 1.21 第 21 章 全书总结：把 Agent 系统做成工程系统

MicroClaw 的实践可以归纳为一句话：让模型做决策，让运行时做约束，让系统做记忆，让运维做闭环。

如果你要从这本书带走三件事，建议是：

1. 先做执行闭环，再做能力扩张。
2. 先做可观测和可恢复，再做自动化规模化。
3. 先做安全边界，再做体验提速。

从项目代码和文档可以看到，MicroClaw 的价值并不在“某个炫目的单点特性”，而在多个朴素机制的长期配合：

1. 统一 Agent Loop，避免平台分叉。
2. 工具抽象统一，策略检查前置。
3. 安全分层治理，审批与隔离并行。
4. 记忆分层治理，提取与注入可观测。
5. 调度与 DLQ 闭环，支持长期自动化。
6. runbook + SLO，把经验变成制度。

当我们讨论“从零构建”时，真正难的从来不是写出第一个可运行版本，而是把它变成一个长期可维护、可治理、可演进的系统。Micro-Claw 提供了一个现实可行的工程样本。

愿这本书成为你的起点：不是复制实现细节，而是掌握背后的设计原则，并在你的系统里落地成自己的方法论。

## 1.22 第 22 章 实战练习题库： 从入门到进阶的 60 组 工程训练

### 1.22.1 22.1 使用方法

本章提供 60 组训练任务。每组都按同一结构给出：

1. 任务背景。
2. 训练目标。
3. 建议步骤。
4. 验收标准。
5. 延伸思考。

建议做法：每次练习结束后，把执行路径写入 todo 与复盘文档，再对照本书前文章节检查是否遵守

了“计划先行、边界优先、可观测收敛”的原则。

## 1.22.2 22.2 训练题库

### 1.22.2.1 练习 1: Agent Loop 收敛 场景演练

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。

2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。



4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### 1.22.2.2 练习 2：工具风险控制 场景演练

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。

2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

### **1.22.2.3 练习 3: Sandbox 隔离场景演练**

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：

1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人



工确认？请给出“自动化优先级列表”，并说明你的排序依据。

#### 1.22.2.4 练习 4：记忆质量 场景演练

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。

4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

#### **1.22.2.5 练习 5：调度恢复 场景演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。

2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

#### **1.22.2.6 练习 6: 多渠道一致性 场景演练**

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：



1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人

工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### 1.22.2.7 练习 7：配置调优 场景演练

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。

4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

#### **1.22.2.8 练习 8: Web 控制面 场景 演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。



2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

### **1.22.2.9 练习 9: 权限治理 场景演练**

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：

1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人

工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### 1.22.2.10 练习 10：发布回滚 场景 演练

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。

4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### **1.22.2.11 练习 11: Agent Loop 收敛 场景演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。



- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。

2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

### **1.22.2.12 练习 12: 工具风险控制 场景演练**

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：

1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人

工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### **1.22.2.13 练习 13: Sandbox 隔离场景演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。



4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

#### **1.22.2.14 练习 14：记忆质量 场景演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。

2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

### **1.22.2.15 练习 15: 调度恢复 场景 演练**

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：

1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人



工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### **1.22.2.16 练习 16：多渠道一致性 场景演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。

4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### 1.22.2.17 练习 17：配置调优 场景演练

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。

2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

### **1.22.2.18 练习 18: Web 控制面 场景演练**

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：



1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人

工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### 1.22.2.19 练习 19：权限治理 场景 演练

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。

4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### **1.22.2.20 练习 20：发布回滚 场景演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。



2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

### 1.22.2.21 练习 21: Agent Loop 收敛 场景演练

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：

1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人

工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### 1.22.2.22 练习 22：工具风险控制 场景演练

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。

4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### **1.22.2.23 练习 23: Sandbox 隔离场景演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。



- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。

2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

#### **1.22.2.24 练习 24: 记忆质量 场景 演练**

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：

1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人

工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### 1.22.2.25 练习 25：调度恢复 场景 演练

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。



4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### **1.22.2.26 练习 26：多渠道一致性 场景演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。

2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

### **1.22.2.27 练习 27: 配置调优 场景演练**

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：

1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人



工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### 1.22.2.28 练习 28: Web 控制面 场景演练

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。

4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### **1.22.2.29 练习 29：权限治理 场景演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。

2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

### **1.22.2.30 练习 30: 发布回滚 场景演练**

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：



1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人

工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### 1.22.2.31 练习 31: Agent Loop 收敛 场景演练

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。

4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### **1.22.2.32 练习 32：工具风险控制 场景演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。



2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

### **1.22.2.33 练习 33: Sandbox 隔离场景演练**

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：

1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人

工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### 1.22.2.34 练习 34：记忆质量 场景 演练

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。

4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### **1.22.2.35 练习 35：调度恢复 场景演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。



- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。

2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

### **1.22.2.36 练习 36: 多渠道一致性 场景演练**

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：

1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人

工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### 1.22.2.37 练习 37：配置调优 场景 演练

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。



4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### **1.22.2.38 练习 38: Web 控制面 场景演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。

2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

### **1.22.2.39 练习 39: 权限治理 场景演练**

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：

1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人



工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### 1.22.2.40 练习 40：发布回滚 场景 演练

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。

4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

#### 1.22.2.41 练习 41: Agent Loop 收敛 场景演练

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下, 对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。

2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

#### **1.22.2.42 练习 42: 工具风险控制 场景演练**

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：



1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人

工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### **1.22.2.43 练习 43: Sandbox 隔离场景演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。

4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

#### **1.22.2.44 练习 44：记忆质量 场景 演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。



2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

#### **1.22.2.45 练习 45: 调度恢复 场景 演练**

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：

1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人

工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### 1.22.2.46 练习 46：多渠道一致性 场景演练

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。

4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

#### **1.22.2.47 练习 47：配置调优 场景演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。



- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。

2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

### 1.22.2.48 练习 48: Web 控制面 场景演练

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：

1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人

工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### **1.22.2.49 练习 49：权限治理 场景 演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。



4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### **1.22.2.50 练习 50：发布回滚 场景演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。

2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

### 1.22.2.51 练习 51: Agent Loop 收敛 场景演练

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：

1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人



工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### 1.22.2.52 练习 52：工具风险控制 场景演练

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。

4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### **1.22.2.53 练习 53: Sandbox 隔离场景演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。

2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

#### **1.22.2.54 练习 54: 记忆质量 场景 演练**

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：



1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人

工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### 1.22.2.55 练习 55：调度恢复 场景 演练

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。

4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### **1.22.2.56 练习 56：多渠道一致性 场景演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。



2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

### **1.22.2.57 练习 57: 配置调优 场景演练**

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：

1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人

工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### 1.22.2.58 练习 58: Web 控制面 场景演练

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。
- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。

3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
    1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
    2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
    3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。

4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。

4. 最终结论包含可复现步骤，不依赖个人记忆。
- 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### **1.22.2.59 练习 59：权限治理 场景演练**

- 任务背景：你接手一个运行中的 MicroClaw 部署，团队反馈“功能基本可用，但在复杂任务、异常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。



- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：
  1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
  2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。

3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
  4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
  5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。
  6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。

2. 关键工具调用结果可追踪, 失败信息没有被掩盖。
  3. 至少形成一条可执行改进建议 (配置、策略或流程层面)。
  4. 最终结论包含可复现步骤, 不依赖个人记忆。
- 延伸思考: 如果把这个练习放进 CI/CD 或周度运维节奏, 应该把哪些检查自动化, 哪些仍保留人工确认? 请给出“自动化优先级列表”, 并说明你的排序依据。

### **1.22.2.60 练习 60: 发布回滚 场景演练**

- 任务背景: 你接手一个运行中的 MicroClaw 部署, 团队反馈“功能基本可用, 但在复杂任务、异

常恢复和安全边界上偶尔出现不稳定”。你需要在不破坏现有业务连续性的前提下，对系统进行一轮可验证的工程加固。

- 训练目标：
  1. 将当前问题转化为可执行的 todo 计划，并保持状态同步。
  2. 明确本练习涉及的工具、权限、隔离边界与观测指标。
  3. 在执行过程中记录关键决策点，避免“黑箱完成”。
  4. 输出可复用的检查清单与回滚方案。
- 建议步骤：

1. 先调用 todo 机制，把任务分为“诊断、执行、验证、复盘”四阶段。
2. 用最小工具集完成第一轮诊断，优先收集证据，不急于修改。
3. 明确风险等级：哪些动作属于高风险、是否需要审批或沙箱强制。
4. 执行改动后立刻做对照验证，至少覆盖成功路径和失败路径。
5. 如果出现异常，按 runbook 做故障分级与处置，而不是临时拍脑袋重试。

6. 完成后输出“做了什么、为什么这么做、还存在哪些残余风险”。
- 验收标准：
    1. 有完整 todo 轨迹，且只有一个 in\_progress 任务。
    2. 关键工具调用结果可追踪，失败信息没有被掩盖。
    3. 至少形成一条可执行改进建议（配置、策略或流程层面）。
    4. 最终结论包含可复现步骤，不依赖个人记忆。
  - 延伸思考：如果把这个练习放进 CI/CD 或周度运维节奏，应该把哪些检查自动化，哪些仍保留人

工确认？请给出“自动化优先级列表”，并说明你的排序依据。

### **1.22.3 22.3 题库使用建议**

1. 初级阶段：每周完成 2~3 题，重点练“计划与执行同步”。
2. 中级阶段：按主题分组练习，重点练“跨模块联动”。
3. 高级阶段：把练习改造成真实值班演练，重点练“事故处置速度与质量”。

### **1.22.4 22.4 本章小结**

训练的价值不在于做完多少题，而在于形成稳定的方法论：先定义边界，再收集证据，再执行最小变更，最后完成复盘闭环。

## 1.23 第 23 章 设计决策备忘录：80 条 ADR 风格取舍记录

### 1.23.1 23.1 阅读说明

本章采用轻量 ADR (Architecture Decision Record) 体例。每条记录包含：

1. 决策主题。
2. 背景问题。
3. 方案选择。
4. 主要收益。
5. 代价与后续动作。

这些记录可直接作为团队内部架构评审模板。



## 1.23.2 23.2 决策记录

### 1.23.2.1 ADR-1: 共享 Agent Loop

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“共享 Agent Loop”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 减少平台分叉, 让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束, 降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险：
  1. 需要更多约束治理, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。

3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作:
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准:
    1. 关键路径行为在多渠道保持一致。

2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

#### 1.23.2.2 ADR-2: 工具策略前置校验

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“工具策略前置校验”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 提升故障可诊断性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 可能增加首配复杂度, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.3 ADR-3: 高风险审批门

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单

点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“高风险审批门”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 控制高风险动作，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。



3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 会引入交互摩擦, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

#### 1.23.2.4 ADR-4: sandbox fail-closed

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“sandbox fail-closed”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新

功能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 避免隐式降级，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 要求更完整监控，短期内学习成本会上升。

2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.5 ADR-5: memory token budget**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择：采用“memory token budget”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 平衡记忆与上下文，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 需要持续调优，短期内学习成本会上升。
  2. 某些历史路径需要迁移，存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓，因为需要补充验证和回归。
  4. 若缺乏执行纪律，策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。



4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

#### 1.23.2.6 ADR-6: todo 计划先行

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单

点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“todo 计划先行”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 提升任务可追踪性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 要求团队纪律, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

#### 1.23.2.7 ADR-7: scheduler + DLQ

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“scheduler + DLQ”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功

能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 保证自动化恢复，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 要维护恢复流程，短期内学习成本会上升。

2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

#### 1.23.2.8 ADR-8: Web scope 化授权

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。



- 方案选择：采用“Web scope 化授权”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 降低管理面越权，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 需要权限模型演进, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

#### 1.23.2.9 ADR-9: 共享 Agent Loop

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“共享 Agent Loop”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 减少平台分叉，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 需要更多约束治理, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.10 ADR-10: 工具策略前置校验

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“工具策略前置校验”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功

能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 提升故障可诊断性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 可能增加首配复杂度，短期内学习成本会上升。



2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.11 ADR-11: 高风险审批门**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“高风险审批门”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 控制高风险动作，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 会引入交互摩擦，短期内学习成本会上升。
  2. 某些历史路径需要迁移，存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓，因为需要补充验证和回归。
  4. 若缺乏执行纪律，策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.12 ADR-12: sandbox fail-closed**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“sandbox fail-closed”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 避免隐式降级，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 要求更完整监控, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。



4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.13 ADR-13: memory token budget

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“memory token budget”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新

功能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 平衡记忆与上下文，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 需要持续调优，短期内学习成本会上升。

2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.14 ADR-14: todo 计划先行**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“todo 计划先行”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 提升任务可追踪性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 要求团队纪律, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.15 ADR-15: scheduler + DLQ**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“scheduler + DLQ”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 保证自动化恢复，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。



3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 要维护恢复流程, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.16 ADR-16: Web scope 化授权

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“Web scope 化授权”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功

能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 降低管理面越权，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 需要权限模型演进，短期内学习成本会上升。

2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.17 ADR-17: 共享 Agent Loop

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“共享 Agent Loop”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 减少平台分叉，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 需要更多约束治理, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。



4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.18 ADR-18: 工具策略前置校验**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“工具策略前置校验”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 提升故障可诊断性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 可能增加首配复杂度, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.19 ADR-19: 高风险审批门

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“高风险审批门”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 控制高风险动作, 让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束, 降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险：
  1. 会引入交互摩擦, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。

3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作:
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准:
    1. 关键路径行为在多渠道保持一致。

2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.20 ADR-20: sandbox fail-closed**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。



- 方案选择： 采用“sandbox fail-closed”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 避免隐式降级，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 要求更完整监控, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.21 ADR-21: memory to-ken budget**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择：采用“memory token budget”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 平衡记忆与上下文，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 需要持续调优, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.22 ADR-22: todo 计划先行

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“todo 计划先行”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能

必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 提升任务可追踪性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 要求团队纪律，短期内学习成本会上升。



2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

### **1.23.2.23 ADR-23: scheduler + DLQ**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“scheduler + DLQ”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 保证自动化恢复，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 要维护恢复流程, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.24 ADR-24: Web scope 化授权**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择：采用“Web scope 化授权”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 降低管理面越权，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 需要权限模型演进, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。



4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.25 ADR-25: 共享 Agent Loop

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“共享 Agent Loop”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功

能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 减少平台分叉，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 需要更多约束治理，短期内学习成本会上升。

2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

### **1.23.2.26 ADR-26: 工具策略前置校验**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“工具策略前置校验”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 提升故障可诊断性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 可能增加首配复杂度, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.27 ADR-27: 高风险审批门

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单

点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“高风险审批门”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 控制高风险动作，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。



3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 会引入交互摩擦, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

### **1.23.2.28 ADR-28: sandbox fail-closed**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“sandbox fail-closed”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新

功能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 避免隐式降级，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 要求更完整监控，短期内学习成本会上升。

2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

### **1.23.2.29 ADR-29: memory token budget**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择：采用“memory token budget”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 平衡记忆与上下文，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 需要持续调优，短期内学习成本会上升。
  2. 某些历史路径需要迁移，存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓，因为需要补充验证和回归。
  4. 若缺乏执行纪律，策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。



4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.30 ADR-30: todo 计划先行

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“todo 计划先行”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 提升任务可追踪性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 要求团队纪律, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.31 ADR-31: scheduler + DLQ

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“scheduler + DLQ”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功

能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 保证自动化恢复，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 要维护恢复流程，短期内学习成本会上升。

2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

### **1.23.2.32 ADR-32: Web scope 化授权**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。



- 方案选择：采用“Web scope 化授权”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 降低管理面越权，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 需要权限模型演进, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.33 ADR-33: 共享 Agent Loop

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“共享 Agent Loop”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 减少平台分叉，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 需要更多约束治理, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.34 ADR-34: 工具策略前置校验

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“工具策略前置校验”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功

能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 提升故障可诊断性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 可能增加首配复杂度，短期内学习成本会上升。



2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.35 ADR-35: 高风险审批门**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“高风险审批门”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 控制高风险动作，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 会引入交互摩擦, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

### **1.23.2.36 ADR-36: sandbox fail-closed**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“sandbox fail-closed”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 避免隐式降级，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 要求更完整监控, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。



4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.37 ADR-37: memory token budget

- 背景问题：在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择：采用“memory token budget”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新

功能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 平衡记忆与上下文，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 需要持续调优，短期内学习成本会上升。

2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.38 ADR-38: todo 计划先行

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“todo 计划先行”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 提升任务可追踪性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 要求团队纪律, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

### **1.23.2.39 ADR-39: scheduler + DLQ**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“scheduler + DLQ”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 保证自动化恢复，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。



3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 要维护恢复流程, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

#### 1.23.2.40 ADR-40: Web scope 化授权

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“Web scope 化授权”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功

能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 降低管理面越权，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 需要权限模型演进，短期内学习成本会上升。

2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.41 ADR-41: 共享 Agent Loop**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“共享 Agent Loop”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 减少平台分叉，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 需要更多约束治理, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。



4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.42 ADR-42: 工具策略前置校验**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“工具策略前置校验”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 提升故障可诊断性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 可能增加首配复杂度, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

### **1.23.2.43 ADR-43: 高风险审批门**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“高风险审批门”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 控制高风险动作, 让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束, 降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险：
  1. 会引入交互摩擦, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。

3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作:
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准:
    1. 关键路径行为在多渠道保持一致。

2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.44 ADR-44: sandbox fail-closed**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。



- 方案选择： 采用“sandbox fail-closed”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 避免隐式降级，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 要求更完整监控, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

#### 1.23.2.45 ADR-45: memory token budget

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择：采用“memory token budget”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 平衡记忆与上下文，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 需要持续调优, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.46 ADR-46: todo 计划先行

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“todo 计划先行”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能

必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 提升任务可追踪性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 要求团队纪律，短期内学习成本会上升。



2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.47 ADR-47: scheduler + DLQ**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“scheduler + DLQ”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 保证自动化恢复，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 要维护恢复流程, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.48 ADR-48: Web scope 化授权**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择：采用“Web scope 化授权”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 降低管理面越权，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 需要权限模型演进, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。



4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.49 ADR-49: 共享 Agent Loop

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“共享 Agent Loop”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功

能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 减少平台分叉，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 需要更多约束治理，短期内学习成本会上升。

2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.50 ADR-50: 工具策略前置 校验**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“工具策略前置校验”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 提升故障可诊断性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 可能增加首配复杂度, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.51 ADR-51: 高风险审批门

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单

点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“高风险审批门”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 控制高风险动作，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。



3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 会引入交互摩擦, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

### **1.23.2.52 ADR-52: sandbox fail-closed**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“sandbox fail-closed”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新

功能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 避免隐式降级，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 要求更完整监控，短期内学习成本会上升。

2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.53 ADR-53: memory token budget**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择：采用“memory token budget”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 平衡记忆与上下文，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 需要持续调优, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。



4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.54 ADR-54: todo 计划先行

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“todo 计划先行”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 提升任务可追踪性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 要求团队纪律, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

### **1.23.2.55 ADR-55: scheduler + DLQ**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“scheduler + DLQ”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功

能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 保证自动化恢复，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 要维护恢复流程，短期内学习成本会上升。

2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.56 ADR-56: Web scope 化授权**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。



- 方案选择：采用“Web scope 化授权”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 降低管理面越权，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 需要权限模型演进, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.57 ADR-57: 共享 Agent Loop

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“共享 Agent Loop”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 减少平台分叉，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 需要更多约束治理, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

### **1.23.2.58 ADR-58: 工具策略前置校验**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“工具策略前置校验”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功

能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 提升故障可诊断性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 可能增加首配复杂度，短期内学习成本会上升。



2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

### **1.23.2.59 ADR-59: 高风险审批门**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“高风险审批门”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 控制高风险动作，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 会引入交互摩擦, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.60 ADR-60: sandbox fail-closed**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“sandbox fail-closed”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 避免隐式降级，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 要求更完整监控, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。



4. 团队成员可用同一模板解释该决策与取舍。

#### 1.23.2.61 ADR-61: memory token budget

- 背景问题：在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择：采用“memory token budget”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新

功能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 平衡记忆与上下文，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 需要持续调优，短期内学习成本会上升。

2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

#### 1.23.2.62 ADR-62: todo 计划先行

- 背景问题：在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“todo 计划先行”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 提升任务可追踪性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 要求团队纪律，短期内学习成本会上升。
  2. 某些历史路径需要迁移，存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓，因为需要补充验证和回归。
  4. 若缺乏执行纪律，策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.63 ADR-63: scheduler + DLQ**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“scheduler + DLQ”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 保证自动化恢复，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。



3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 要维护恢复流程, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.64 ADR-64: Web scope 化授权

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“Web scope 化授权”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功

能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 降低管理面越权，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 需要权限模型演进，短期内学习成本会上升。

2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

### **1.23.2.65 ADR-65: 共享 Agent Loop**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“共享 Agent Loop”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 减少平台分叉，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 需要更多约束治理, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。



4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.66 ADR-66: 工具策略前置校验**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“工具策略前置校验”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 提升故障可诊断性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 可能增加首配复杂度, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.67 ADR-67: 高风险审批门

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“高风险审批门”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 控制高风险动作, 让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束, 降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险：
  1. 会引入交互摩擦, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。

3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作:
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准:
    1. 关键路径行为在多渠道保持一致。

2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.68 ADR-68: sandbox fail-closed**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。



- 方案选择： 采用“sandbox fail-closed”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 避免隐式降级，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 要求更完整监控, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.69 ADR-69: memory token budget**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择：采用“memory token budget”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 平衡记忆与上下文，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 需要持续调优, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.70 ADR-70: todo 计划先行

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“todo 计划先行”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能

必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 提升任务可追踪性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 要求团队纪律，短期内学习成本会上升。



2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.71 ADR-71: scheduler + DLQ

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“scheduler + DLQ”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 保证自动化恢复，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 要维护恢复流程，短期内学习成本会上升。
  2. 某些历史路径需要迁移，存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓，因为需要补充验证和回归。
  4. 若缺乏执行纪律，策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.72 ADR-72: Web scope 化授权

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择：采用“Web scope 化授权”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 降低管理面越权，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 需要权限模型演进, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。



4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.73 ADR-73: 共享 Agent Loop

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“共享 Agent Loop”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功

能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 减少平台分叉，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 需要更多约束治理，短期内学习成本会上升。

2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

### **1.23.2.74 ADR-74: 工具策略前置 校验**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“工具策略前置校验”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 提升故障可诊断性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 可能增加首配复杂度, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

### **1.23.2.75 ADR-75: 高风险审批门**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单

点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“高风险审批门”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 控制高风险动作，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。



3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 会引入交互摩擦, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.76 ADR-76: sandbox fail-closed

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“sandbox fail-closed”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新

功能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 避免隐式降级，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 要求更完整监控，短期内学习成本会上升。

2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

### **1.23.2.77 ADR-77: memory token budget**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择：采用“memory token budget”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 平衡记忆与上下文，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 需要持续调优，短期内学习成本会上升。
  2. 某些历史路径需要迁移，存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓，因为需要补充验证和回归。
  4. 若缺乏执行纪律，策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。



4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.78 ADR-78: todo 计划先行

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张

力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。

- 方案选择： 采用“todo 计划先行”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 提升任务可追踪性，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。

3. 让测试和观测可以围绕稳定契约建立, 而不是追逐临时行为。
  4. 为后续扩展(渠道、工具、策略)提供统一演进支点。
- 代价与风险:
    1. 要求团队纪律, 短期内学习成本会上升。
    2. 某些历史路径需要迁移, 存在兼容性治理工作量。
    3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
    4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
  - 落地动作:

1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。
  4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。

4. 团队成员可用同一模板解释该决策与取舍。

### 1.23.2.79 ADR-79: scheduler + DLQ

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。
- 方案选择： 采用“scheduler + DLQ”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功

能必须先对齐该策略，再进入实现和发布流程。

- 主要收益：
  1. 保证自动化恢复，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。
- 代价与风险：
  1. 要维护恢复流程，短期内学习成本会上升。

2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
    1. 在代码层加入强约束与结构化错误返回。
    2. 在文档层同步更新配置说明与 runbook。
    3. 在测试层增加回归用例覆盖关键分支。
    4. 在运维层增加对应指标与告警阈值。
  - 验证标准：

1. 关键路径行为在多渠道保持一致。
2. 异常发生时可在固定时间内定位到模块层级。
3. 变更后未出现同类回归或同类事故显著下降。
4. 团队成员可用同一模板解释该决策与取舍。

#### **1.23.2.80 ADR-80: Web scope 化授权**

- 背景问题： 在持续迭代过程中，团队发现当前系统在“功能可用”与“长期可维护”之间存在张力。短期内的快速补丁能解决单点问题，但会逐渐侵蚀核心一致性，最终带来跨模块行为漂移和运维复杂度上升。



- 方案选择：采用“Web scope 化授权”作为本阶段主策略，并将相关规则放入运行时主路径，而不是仅写入文档约定。所有新功能必须先对齐该策略，再进入实现和发布流程。
- 主要收益：
  1. 降低管理面越权，让系统行为更可预测。
  2. 把隐性经验沉淀为显性约束，降低人员变动影响。
  3. 让测试和观测可以围绕稳定契约建立，而不是追逐临时行为。
  4. 为后续扩展（渠道、工具、策略）提供统一演进支点。

- 代价与风险：
  1. 需要权限模型演进, 短期内学习成本会上升。
  2. 某些历史路径需要迁移, 存在兼容性治理工作量。
  3. 发布节奏可能在初期放缓, 因为需要补充验证和回归。
  4. 若缺乏执行纪律, 策略可能停留在“名义存在”。
- 落地动作：
  1. 在代码层加入强约束与结构化错误返回。
  2. 在文档层同步更新配置说明与 runbook。
  3. 在测试层增加回归用例覆盖关键分支。

4. 在运维层增加对应指标与告警阈值。
- 验证标准：
    1. 关键路径行为在多渠道保持一致。
    2. 异常发生时可在固定时间内定位到模块层级。
    3. 变更后未出现同类回归或同类事故显著下降。
    4. 团队成员可用同一模板解释该决策与取舍。

### **1.23.3 23.3 决策管理建议**

1. 每个季度挑选 5 条关键 ADR 做有效性复审。
2. 若决策已失效，要显式标注“废弃”并给替代方案。

3. ADR 变更必须关联测试与文档改动，避免纸面决策。
4. 对高风险 ADR 增加“演练证明”要求。

### **1.23.4 23.4 本章小结**

把设计决策写下来，不会自动提高系统质量，但会显著提高团队对质量的可重复执行能力。这是 Agent 系统从“个人技巧”走向“组织能力”的关键一步。

## **1.24 附录**

### **1.24.1 附录 A 术语表**

1. Agent Loop：由模型决策与工具执行组成的迭代闭环。

2. stop\_reason: 模型本轮停止原因，驱动状态转移。
3. ToolDefinition: 供模型调用的工具声明（名称、描述、输入模式）。
4. ToolResult: 工具执行标准返回结构。
5. ToolRisk: 工具风险等级（low/medium/high）。
6. ToolExecutionPolicy: 工具执行位置策略（host-only/sandbox-only/dual）。
7. Approval Gate: 高风险操作审批门。
8. Control Chat: 可跨 chat 执行操作的受信会话。

9. WorkingDir Isolation: 工具工作目录隔离模式 (shared/chat)。
10. SandboxRouter: 决定宿主/容器执行路径的路由器。
11. Fail-open: 隔离依赖不可用时回退执行。
12. Fail-closed: 隔离依赖不可用时拒绝执行。
13. AGENTS.md: 文件记忆载体。
14. Structured Memory: 数据库结构化记忆。
15. Reflector: 后台记忆提取任务。
16. Memory Injection: 把记忆注入系统提示词的过程。

- 17. Token Budget: 注入内容的估算 token 上限。
- 18. Session Resume: 会话持久化恢复机制。
- 19. Context Compaction: 上下文压缩归档机制。
- 20. Todo Orchestration: 多步骤任务的计划-执行同步机制。
- 21. Scheduler: 定时任务执行器。
- 22. DLQ: 失败任务死信队列。
- 23. Hook: 生命周期策略扩展点。
- 24. BeforeLLMCall: 模型调用前 hook 事件。
- 25. BeforeToolCall: 工具调用前 hook 事件。
- 26. AfterToolCall: 工具调用后 hook 事件。

- 27. Self-check: 配置风险自检接口。
- 28. SLO: 服务级目标指标集合。
- 29. Burn Alert: SLO 消耗告警。
- 30. Capability Flags: 模型/提供商能力标识。

## 1.24.2 附录 B 工具清单与风险等级速查

### 1.24.2.1 B.1 内建工具列表（当前）

- 1. `activate_skill`
- 2. `bash`
- 3. `browser`
- 4. `cancel_scheduled_task`
- 5. `edit_file`
- 6. `export_chat`
- 7. `get_task_history`



8. glob
9. grep
10. list\_scheduled\_task\_dlq
11. list\_scheduled\_tasks
12. pause\_scheduled\_task
13. read\_file
14. read\_memory
15. replay\_scheduled\_task\_dlq
16. resume\_scheduled\_task
17. schedule\_task
18. send\_message
19. structured\_memory\_delete
20. structured\_memory\_search
21. structured\_memory\_update
22. sub\_agent
23. sync\_skills
24. todo\_read

- 25. `todo_write`
- 26. `web_fetch`
- 27. `web_search`
- 28. `write_file`
- 29. `write_memory`

#### 1.24.2.2 B.2 风险分级速查

- High

- 1. `bash`

- Medium

- 1. `write_file`
- 2. `edit_file`
- 3. `write_memory`
- 4. `send_message`
- 5. `sync_skills`
- 6. `schedule_task`

7. `pause_scheduled_task`
8. `resume_scheduled_task`
9. `cancel_scheduled_task`
10. `replay_scheduled_task_dlog`
11. `structured_memory_delete`
12. `structured_memory_update`

- Low

1. 其余未列入 high/medium 的工具默认 low。

### 1.24.2.3 B.3 执行策略速查（当前基线）

1. `bash`: dual
2. `write_file`: host-only
3. `edit_file`: host-only
4. 其余工具: host-only

#### 1.24.2.4 B.4 子代理允许工具（受限工具集）

1. bash
2. browser
3. read\_file
4. write\_file
5. edit\_file
6. glob
7. grep
8. read\_memory
9. web\_fetch
10. web\_search
11. activate\_skill
12. structured\_memory\_search

#### 1.24.2.5 B.5 子代理默认禁用能力

1. send\_message

2. write\_memory
3. schedule\_task 及其管理工具
4. export\_chat
5. sub\_agent (防递归)

### 1.24.3 附录 C 默认配置项速查表

#### 1.24.3.1 C.1 LLM 与循环控制

1. llm\_provider: anthropic
2. max\_tokens: 8192
3. max\_tool\_iterations: 100
4. max\_history\_messages: 50
5. max\_session\_messages: 40
6. compact\_keep\_recent: 20
7. compaction\_timeout\_secs: 180
8. memory\_token\_budget: 1500

### 1.24.3.2 C.2 路径与运行目录

1. data\_dir: 默认 ~/.microclaw
2. working\_dir: 默 认  
~/.microclaw/working\_dir
3. working\_dir\_isolation: chat

### 1.24.3.3 C.3 沙箱默认值

1. sandbox.mode: off
2. sandbox.backend: auto
3. sandbox.image: ubuntu:25.10
4. sandbox.container\_prefix:  
microclaw-sandbox
5. sandbox.no\_network: true
6. sandbox.require\_runtime:  
false
7. sandbox.mount\_allowlist\_path:  
null
8. sandbox.memory\_limit: null

- 9. `sandbox.cpu_quota: null`
- 10. `sandbox.pids_limit: null`

#### **1.24.3.4 C.4 Web 默认值**

- 1. `web_enabled: true`
- 2. `web_host: 127.0.0.1`
- 3. `web_port: 10961`
- 4. `web_max_inflight_per_session:`  
`2`
- 5. `web_max_requests_per_window:`  
`8`
- 6. `web_rate_window_seconds: 10`
- 7. `web_run_history_limit: 512`
- 8. `web_session_idle_ttl_seconds:`  
`300`

#### **1.24.3.5 C.5 调度与反思默认值**

- 1. `timezone: UTC`

2. reflector\_enabled: true
3. reflector\_interval\_mins: 15

### 1.24.3.6 C.6 超时预算默认值

1. default\_tool\_timeout\_secs: 30
2. default\_mcp\_request\_timeout\_secs: 120

### 1.24.3.7 C.7 ClawHub 与语音默认值

1. clawhub\_registry: <https://clawhub.ai>
2. clawhub\_agent\_tools\_enabled: true
3. voice\_provider: openai

### 1.24.3.8 C.8 配置调优备忘

1. 先确定部署画像，再调参数。



2. 先调整超时和限流, 再调整模型参数。
3. 先开启可观测, 再扩大自动化能力。
4. 高频变更参数应纳入变更审计。
5. 所有安全相关参数都应在自检接口中回看。

## 1.24.4 附录 D 参考资料与代码索引

### 1.24.4.1 D.1 项目与文档

1. 项目仓库: <https://github.com/microclaw/microclaw>
2. 官网: <https://microclaw.ai>
3. 作者: <https://github.com/everettjf>
4. 关键文档目录: `/Users/eevv/focus/microclaw/docs`

5. 网站内容目录： /Users/eevv/  
focus/microclaw/website

#### 1.24.4.2 D.2 关键源码索引

1. Agent Loop: src/  
agent\_engine.rs
2. 运行时装配: src/runtime.rs
3. 工具注册与策略校验: src/  
tools/mod.rs
4. 子代理工具: src/tools/  
sub\_agent.rs
5. 调度器与反思器: src/  
scheduler.rs
6. 配置定义: src/config.rs
7. 沙箱路由与后端 :  
crates/microclaw-tools/src/  
sandbox.rs

8. 工 具 运 行 时 策 略 :  
crates/microclaw-tools/src/  
runtime.rs
9. Web 配 置 自 检 : src/web/  
config.rs

### 1.24.4.3 D.3 关键 RFC 与运维文档

1. Auth 模型 : docs/rfcs/0001-  
authn-authz-model.md
2. Hooks 模型 : docs/rfcs/0002-  
hooks-event-model.md
3. 运行手册 : docs/operations/  
runbook.md
4. 执 行 模 型 : docs/security/  
execution-model.md
5. 稳定性计划 : docs/operations/  
stability-plan-2026-q1.md

#### 1.24.4.4 D.4 博客索引

1. `/website/blog/2026-02-07-building-microclaw.md`
2. `/website/blog/2026-02-14-built-with-rust-microclaw-runtime.md`
3. `/website/blog/2026-02-19-microclaw-february-updates.md`